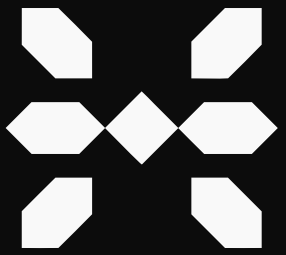
HOOX · PYNE · DOCUMENTATION



EDITOR INTEGRATORS · IDE AUTHORS

LANGUAGE SERVER MANUAL

LSP features, builtin metadata, VS Code, editor clients

Open-source Pine Script evaluation framework

CONTENTS

01 Overview

02 Architecture

03 Diagnostics

04 Completion

05 Hover

06 Navigation

07 Linked Editing

08 Formatting

09 Semantic Tokens

10 Inlay Hints

11 Document Link

12 Document Color

13 Folding

14 Selection Range

15 Builtin Metadata

16 Vscode Extension

17 Clients

OVERVIEW

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Language Server" description: "PYNE Language Server Protocol — diagnostics, completion, hover, formatting for .pyne / .pine and editor clients."

LANGUAGE SERVER

The PYNE **Language Server** is the editor-facing half of the evaluation stack: a pygls process that speaks LSP over STDIO (or TCP in tooling), parses Pine with the same ANTLR4 → ASDL pipeline as the runtime, and never requires a proprietary chart host. First-class file associations include `.pyne` (product extension) plus `.pine` / `.pinev5` / `.pinev6` / `.pinescript`.

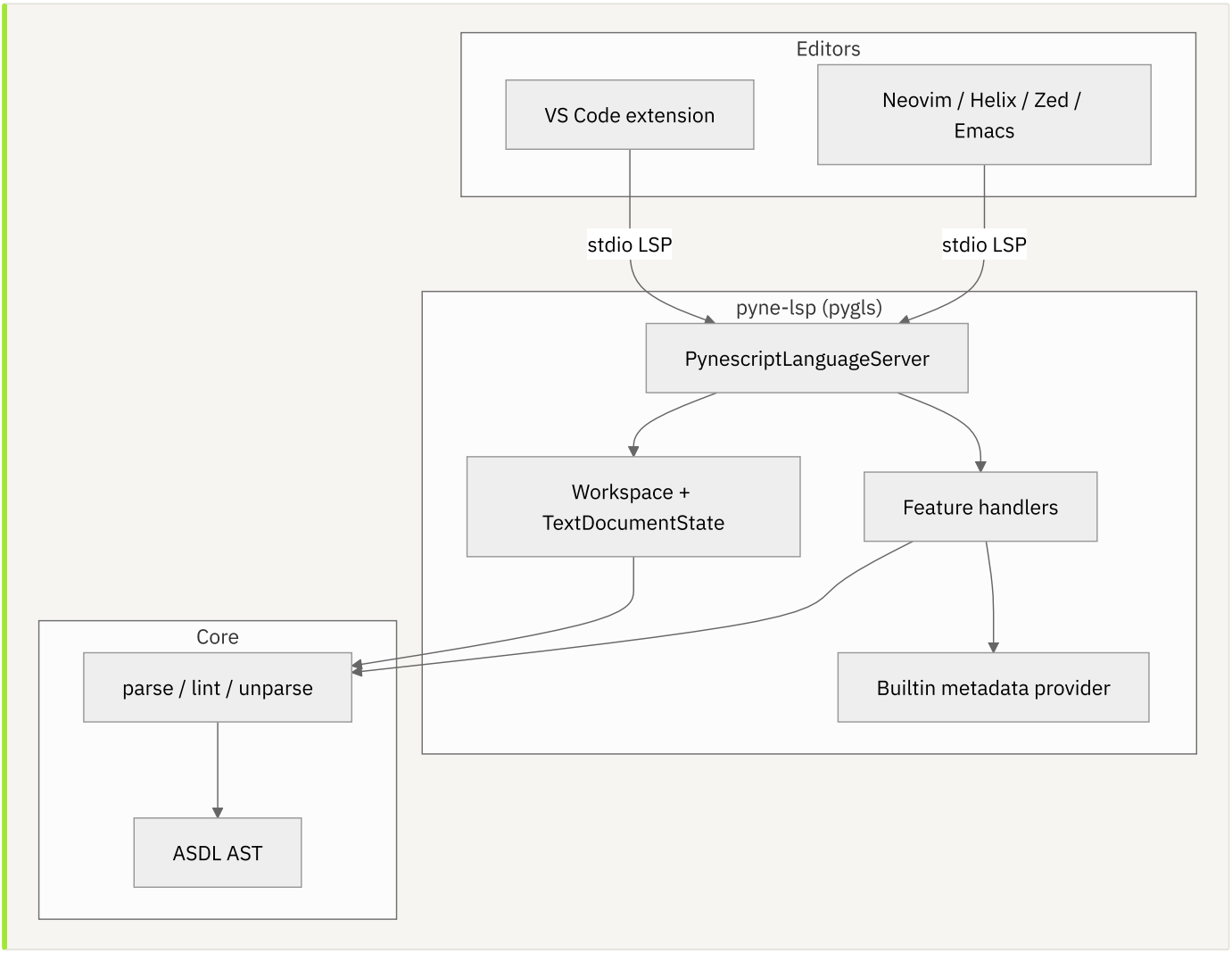
Abstract

Editors want *static* intelligence at typing cadence. Evaluation over OHLCV is *dynamic* and bar-loop expensive. PYNE therefore splits the stack:

Process	Cadence	Shared substrate
LSP (`pyne-lsp`)	keystroke / open / save	parse, lint, unparse, builtin metadata
Pro API (`backend.app`)	request / batch	parse, evaluate, plots / events / alerts
CLI / library	batch / embed	same AST + evaluator

The LSP publishes diagnostics from the linter and parse errors, completes and hovers from Fernet-ready builtin metadata (plus keywords and user enums), navigates user symbols via AST visitors, emits semantic tokens (full and range) from the cached tree, and formats by round-tripping through the unparser (including on-type after newline). Inlay hints cover builtin parameter names; linked editing, document links, and document color round out the editor surface. Feature modules live under `src/pynescript/langserver/features/` ; the VS Code extension (`hoox-sh.pyne` **0.5.4**) and `clients/` configs are thin protocol hosts — VS Code maps `.pyne` **first**, then legacy Pine suffixes, to language id `pinescript` . The stdio image is **`ghcr.io/hoox-sh/pyne/lsp`** .

Conceptual model



Invariant: open/change/close always re-parse and re-lint the in-memory buffer. Features that need an AST re-parse on demand if the workspace cache is missing source.

Interface surface

Capability	LSP method(s)	Module
Sync	`textDocument/didOpen	didChange
Diagnostics (push + pull)	`publishDiagnostics`, `textDocument/diagnostic`	`workspace.py`, `features/diagnostics.py`
Completion (+ resolve)	`textDocument/completion`, `completionItem/resolve`	`features/completion.py`
Hover	`textDocument/hover`	`features/hover.py`
Signature help	`textDocument/signatureHelp`	`features/signature_help.py`
Definition / references	`textDocument/definition`, `textDocument/references`	`definitions.py`, `references.py`
Highlight / rename	`textDocument/documentHighlight`, `prepareRename`, `rename`	`highlight.py`, `rename.py`
Linked editing	`textDocument/linkedEditingRange`	`features/linked_editing.py`
Symbols	`textDocument/documentSymbol`, `workspace/symbol`	`symbols.py`, `server.py`
Formatting	`textDocument/formatting`, `rangeFormatting`, `onTypeFormatting`	`features/formatting.py`
Inlay hints	`textDocument/inlayHint` (display-only builtin param names)	`features/inlay_hints.py`
Folding	`textDocument/foldingRange`	`features/folding.py`
Selection range	`textDocument/selectionRange`	`features/selection_range.py`
Document color	`textDocument/documentColor`, `colorPresentation`	`features/document_color.py`
Document links	`textDocument/documentLink`, `documentLink/resolve` (tooltip)	`features/document_link.py`
Code action	`textDocument/codeAction`	`convert.py` (Convert-to-v6 + W001/W002/C002 QuickFix)
Semantic tokens	`textDocument/semanticTokens/full`, `textDocument/semanticTokens/range`	`features/semantic_tokens.py`

Declared capabilities: `src/pynescript/langserver/config.py` (`get_server_capabilities`).

Internals

Path	Role
`src/pynescript/langserver/server.py`	`PynescriptLanguageServer`, method registration
`src/pynescript/langserver/workspace.py`	Document map, incremental edits, parse+lint
`src/pynescript/langserver/config.py`	ServerCapabilities
`src/pynescript/langserver/features/*`	Per-method handlers
`src/pynescript/langserver/providers/*`	Metadata + completion builders
`src/pynescript/langserver/__main__.py`	STDIO entry (`pyne-lsp` / alias `pynescript-lsp`)
`vscode-extension/`	Language client + TextMate grammar (`hoox-sh.pyne` 0.5.4)
`clients/`	Neovim, Zed, Emacs snippets (Helix / Sublime are documented, not files)

Console scripts are separate: `pyne` (Click CLI; alias `pynescript`) vs `pyne-lsp` (pygls; alias `pynescript-lsp`). Do not conflate them — see architecture.

Tracks in this tab

1. Architecture — server lifecycle, workspace, capabilities
2. Diagnostics through Selection range — feature manuals
3. Linked editing — same-document live ident ranges
4. Document links — `http(s)` URLs in comments and strings; resolve fills tooltip
5. Document color — `#hex` / `color.*` / literal `color.rgb` swatches
6. Builtin metadata — generation + Fernet `.enc` / `CRYPTO_KEY`
7. VS Code extension — client packaging
8. Clients — Neovim, Zed, Emacs, Helix, Sublime

See also

- Pro API — HTTP evaluate path sharing the same AST
- Evaluate contract
- Linter
- Unparser
- Editors guide (end user)

ARCHITECTURE

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

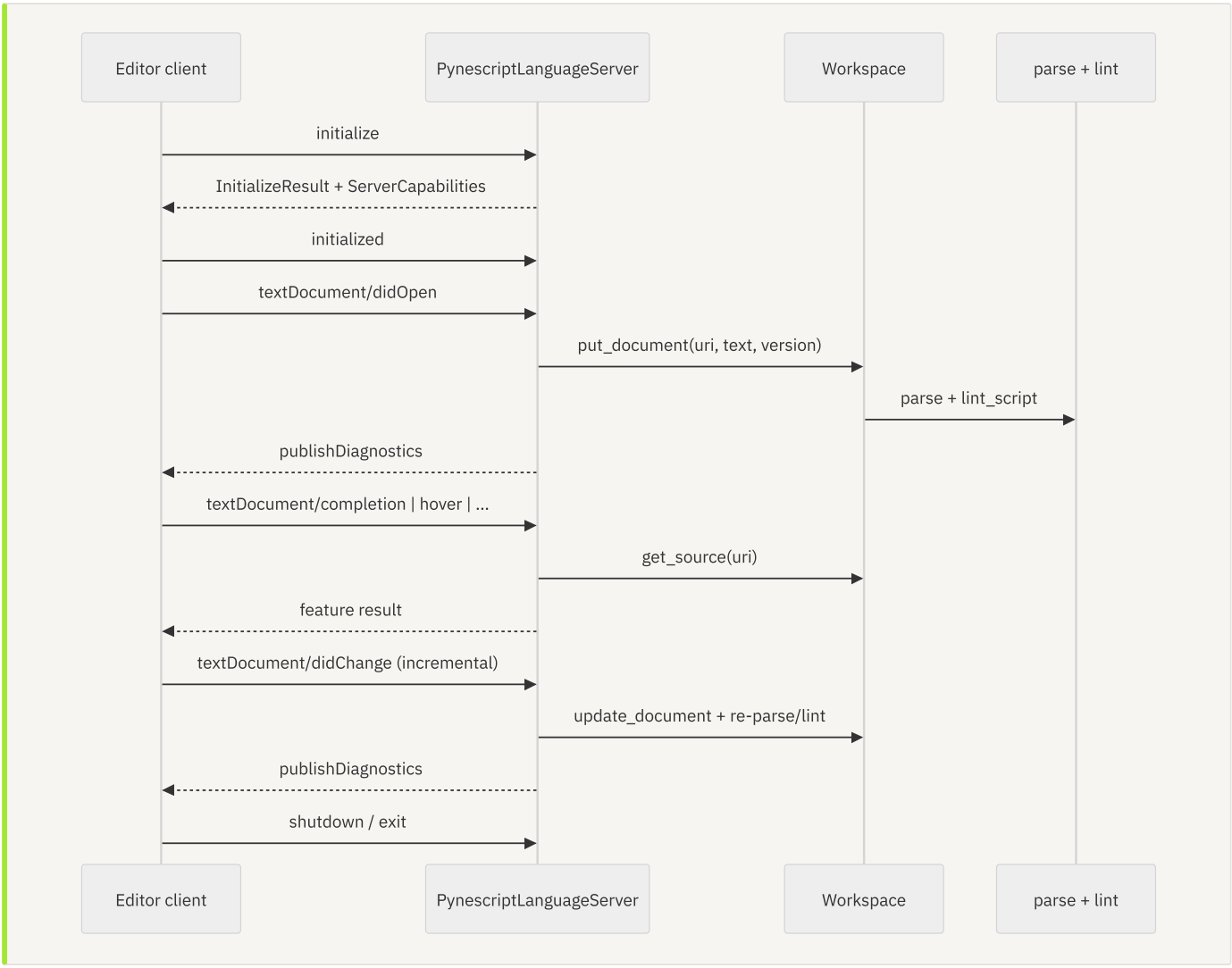
title: "LSP Architecture" description: "PynescriptLanguageServer lifecycle, Workspace document model, capability declaration, and STDIO transport."

LSP ARCHITECTURE

Abstract

The language server is a single-process pygls application. It owns a `Workspace` of open `TextDocumentState` records, re-parses on every mutation, and dispatches LSP methods to pure-ish feature handlers that take `(params, source)` rather than holding server globals. Capabilities are declared once at `initialize`; transport defaults to STDIO for editor integration.

Conceptual model



Interface surface

Process entry

```
# Editable install
pip install -e ".[lsp]"
python -m pynescript.langserver   # same as pyne-lsp / pynescript-lsp
make run-lsp
```

Entry: `src/pynescript/langserver/__main__.py` constructs `PynescriptLanguageServer()` and calls `server.start_io()` (STDIO JSON-RPC).

`pyproject.toml` registers:

```
pyne-lsp = "pynescript.langserver.__main__:main"
pynescript-lsp = "pynescript.langserver.__main__:main"   # alias
```

Server class

`PynescriptLanguageServer` (`server.py`):

- Subclasses `pygls.lsp.server.LanguageServer` with `name="Pynescript"` and `version=__version__` from `src/pynescript/__about__.py` (**0.6.29**). `initialize` reports `serverInfo.name = "Pynescript Language Server"` with the same version.
- Instantiates `self.pine_workspace = Workspace()`.
- Registers handlers in `setup_method_handlers()` via `@self.feature(...)`.

Document sync

Event	Behavior
`didOpen`	`put_document` → parse/lint → push diagnostics
`didChange`	Incremental or full-text apply → re-parse/lint → push diagnostics
`didClose`	Remove document; publish empty diagnostic list
`didSave`	Re-publish diagnostics for current buffer

Sync options (`config.get_server_capabilities`):

- `TextDocumentSyncKind.Incremental`
- `open_close=True`
- `save` with `include_text=True`
- `will_save` / `will_save_wait_until` off

Capability set (declared)

From `config.py` (`get_server_capabilities`):

- Diagnostic provider (`identifier="pynescript-diagnostics"`, `workspace_diagnostics=True`; push diagnostics carry the document version)
- Completion: trigger `"."`, `resolve_provider=True`
- Hover, definition, references, document highlight
- Linked editing (`linked_editing_range_provider=True`) — same-document user ident ranges; keywords / builtin namespaces refused
- Rename with `prepareProvider` (every **open** workspace document that contains the ident; keywords / builtin namespaces refused)
- Document + workspace symbols
- Full + range formatting, plus on-type (`documentOnTypeFormattingProvider`, `first_trigger_character="\n"`)
- Folding ranges (`folding_range_provider=True`) for blocks, type/enum/function bodies, and `//#region`
- Selection ranges (`selection_range_provider=True`) from ident / smallest AST node to enclosing statement and block
- Color provider (`color_provider=True`) — `textDocument/documentColor` + `colorPresentation` for `#hex`, named `color.*`, `color.rgb` / `color.new` / `color()`, and literal `color.from_gradient`
- Document links (`DocumentLinkOptions(resolve_provider=True)`) — `http://` / `https://` URLs in comments (`//`, `/* */`, hash `# …`) and string literals. `documentLink/resolve` fills `tooltip` with the URL when tooltip is unset
- Code action (`QuickFix` + `RefactorRewrite`): Convert to Pine v6, W001 insert `//@version=6`, W002 replace old pragma, C002 format-on-long-line + executeCommand `pynescript.convertToV6`

- Inlay hints (`InlayHintOptions(resolve_provider=False)`): display-only parameter-name hints on builtin calls (`ta.sma(close, 14)` → `source:` / `length:`). No `inlayHint/resolve` , no `data` , no `textEdits`
- Semantic tokens full **and range** (`range=true` , `full=true` ; legend of standard token types/modifiers)
- Signature help (`(` / `,`)

File filters (`get_filter_options`): `*.pine` , `*.pinev5` , `*.pinev6` under language id `pinescript` . **`.pyne` is not in this helper** — VS Code still maps it via the extension contribution (`extensions: .pyne` first).

Internals

Workspace

`Workspace` (`workspace.py`) maps `uri → TextDocumentState` :

```
TextDocumentState
  uri, source, version
  ast | None
  diagnostics: list[LintWarning]
  parse_error, parse_error_line
```

`_parse_and_lint` :

1. `parse(source, filename=uri)` — on success, `lint_script(source, filename=uri)` .
2. On exception: clear AST, store `parse_error` string, extract line via regex `line[:\s]+(\d+)` .

Incremental edits: `_apply_text_edit` splits on `\n` , pads the line list when the range sits at/past EOF (append without a trailing newline), clamps columns, then replaces the range. Out-of-range positions no longer leave a stale buffer. Whole-document change events replace `source` wholesale. Identical text after apply skips re-parse/lint (version still updates).

Position encoding

LSP positions count **UTF-16 code units**; Python strings count **code points**. They differ only for non-BMP characters (emoji, some CJK extensions). The server converts at the boundary:

- request `Position` / `Range` values go UTF-16 → code points before a feature handler runs;
- result ranges go back to UTF-16 on the way out (including nested `WorkspaceEdit` / `Location` objects, which use each document's own lines);
- diagnostics and incremental `didChange` edits convert too;
- `SemanticTokens` columns and lengths are converted before delta encoding.

ASCII/LF buffers skip the recursive conversion pass entirely. Handlers therefore keep using Python string indices internally.

Diagnostics conversion (`_lint_warnings_to_diagnostics`):

- Map severity strings → `DiagnosticSeverity` .
- Source tag `"pyne"` , code from lint rule.
- Append synthetic **E001** error for parse failures at the extracted line.

Feature dispatch pattern

Handlers are functions, not server methods, e.g.:

```
@self.feature(lsp.TEXT_DOCUMENT_COMPLETION)
def text_completion(params):
    source = self.pine_workspace.get_source(params.text_document.uri)
    return completion_feature.handle_completion(params, source)
```

Definition / references / symbols / hover / completion / inlay / semantic tokens (full and range) / linked editing / document links / formatting (including on-type) receive the **version-matched AST** (`tree=doc.valid_ast`) and, where useful, the scope index (`analysis=doc.valid_analysis`). `valid_ast` returns `None` while a debounced parse for a newer buffer is in flight, so features never combine new text with old positions. Pass `tree=None` when the last parse failed. Omit `tree` only in isolated tests (handler then parses from `source`).

Pull diagnostics

- `textDocument/diagnostic` → `RelatedFullDocumentDiagnosticReport` with `result_id=f"{uri}-{version}"` .
- `workspace/diagnostic` → one full report per open document from `get_all_diagnostics()` .

Workspace symbols

`workspace/symbol` walks open documents through the shared document-outline collector (functions/methods, types, enums + members, imports, variables) and then queries `WorkspaceIndex` for closed `.pyne` / `.pine` / `.pinev5` / `.pinev6` / `.pinescript` files under the workspace roots. The closed-file index is bounded (file count/size, ignored build/VCS/dependency directories, mtime cache, 2 s search budget). Filter is case-insensitive substring on `params.query` .

Invariants and edge cases

1. **Parse failure is non-fatal for the process.** AST-dependent features return `None` or `[]` ; diagnostics still surface E001.
2. **Source of truth is the open buffer**, not disk. Save only re-publishes; it does not re-read files.
3. **Incremental sync pads/clamps rather than dropping the edit.** Ranges at EOF grow the line list; columns are clamped to the current line length.
4. **Version** is stored and used in pull-diagnostic `result_id` ; the server does not reject out-of-order versions itself.
5. **Nuitka onefile** embeds providers data and may rely on encrypted metadata — see builtin metadata. Architecture of handlers is identical to the pure-Python path.

Worked example — minimal initialize

Client → server (conceptual):

```
{
  "method": "initialize",
  "params": {
    "capabilities": {},
    "clientInfo": { "name": "example", "version": "1" }
  }
}
```

Server returns `InitializeResult` with `serverInfo.name = "Pynescript Language Server"` and the full `ServerCapabilities` table from `get_server_capabilities()` .

Failure modes

Symptom	Likely cause
No diagnostics on open	Client did not send `didOpen` or language id is not `pinescript`
Completions empty	Metadata file missing/undecryptable; see builtin metadata
Format no-ops	Parse error (handler returns `[]`) or unparsed text equals source
Extension can't spawn server	`pynescript-lsp` not on `PATH`; check VS Code extension

See also

- Diagnostics
- Linked editing
- Formatting — full, range, and on-type
- Semantic tokens — full and range
- Inlay hints — types, param names, resolve
- Document links — `http(s)` URLs; `documentLink/resolve` fills tooltip
- Document color — `#hex` / `color.*` / literal color calls
- Builtin metadata
- VS Code extension
- Nuitka build

DIAGNOSTICS

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

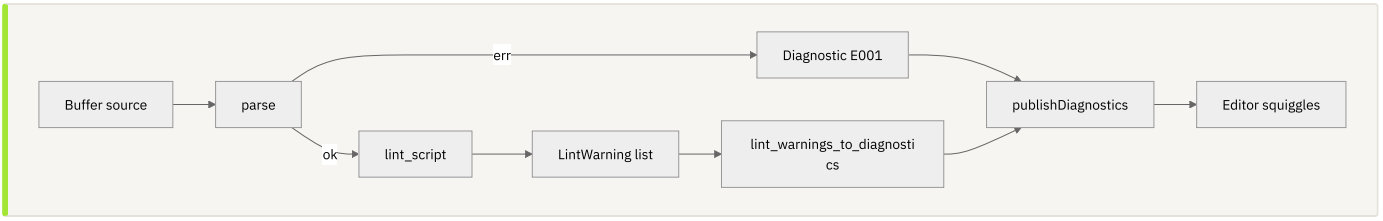
title: "Diagnostics" description: "Push and pull diagnostics: parse errors, lint rules, opt-in semantic checks, severity mapping, and W001/W002/W101/W104/W105/C002 QuickFixes."

DIAGNOSTICS

Abstract

Diagnostics are the primary static feedback channel. The workspace re-parses and re-lints on every open/change/save, converts `LintWarning` objects (and parse failures) into `lsp.Diagnostic`, then **pushes** them with `textDocument/publishDiagnostics`. Clients that support LSP 3.16+ pull diagnostics can also request `textDocument/diagnostic` or `workspace/diagnostic`.

Conceptual model



Interface surface

Push (always on)

Triggered from `server.py` on `didOpen`, `didChange`, `didSave`. Close clears diagnostics (`items=[]`).

Pull

Method	Response
`textDocument/diagnostic`	`RelatedFullDocumentDiagnosticReport` with `kind=full`, `result_id="{uri}-{version}"`
`workspace/diagnostic`	One `WorkspaceFullDocumentDiagnosticReport` per open URI

Capability: `diagnostic_provider` with `identifier="pynescript-diagnostics"`, `inter_file_dependencies=False`.

Diagnostic shape

Field	Value
`range`	Exact token range when the rule supplies `column` / `end_column`; otherwise line start to line end (capped)
`severity`	error / warning / information / hint
`message`	Human-readable lint or parse message
`source`	`"pyne"`
`code`	Lint code (e.g. `W001`) or `E001` for parse errors

The richer conversion in `features/diagnostics.py` also supports:

- `code_description` hrefs for codes starting with `E` / `W` (docs URLs)
- `tags`: `W002` → Deprecated

Push and pull both call `Workspace._lint_warnings_to_diagnostics`, which delegates lint rows to `features/diagnostics.lint_warnings_to_diagnostics` (noise filters + tags) and then appends synthetic **E001** for parse failures.

Internals

Path	Role
`src/pynescript/langserver/workspace.py`	`_parse_and_lint`, E001 append, incremental edits
`src/pynescript/langserver/features/diagnostics.py`	Lint conversion, C001 noise filter, tags, `create_quick_fix`
`src/pynescript/ast/linter.py`	Rule engine producing `LintWarning`

Severity map

```
error        → DiagnosticSeverity.Error
warning      → Warning
info|information → Information
hint         → Hint
(default)    → Warning
```

Version gating

Correctness rules that only apply to v6 are gated on `//@version=6`:

- **W104** (`transp=` / `strategy.* when=`) — valid in v4/v5, removed in v6.
- **W105** (numeric literal in a boolean position) — v6 strictness only.

Unversioned snippets skip both. This prevents the common false-positive flood on v5 scripts while still nudging v6 migrations.

Style diagnostics

C001 (naming), C002 (line length), and C004 (trailing newline) are style-only. The VS Code extension defaults `pynescript.diagnostics.style` to `false`; set it to `true` to show them. Correctness diagnostics are unaffected.

Semantic diagnostics (opt-in)

`pynescript.diagnostics.semantic` (default `false`) enables checks that are decidable from declarations in the current file:

Code	Finding
`S001`	Unknown field/method on a user `type`, or unknown member on a user `enum`
`S002`	User-function call mismatch: too many positional arguments, missing required arguments, unknown named argument

Builtin calls, member access on inferred-but-unknown receivers, and method calls are deliberately skipped. The catalog is incomplete for some host builtins, so under-reporting is preferred to a speculative error. Semantic checks only run when the buffer parses.

Parse errors

On exception from `parse`:

- `ast = None`, `diagnostics = []` (lint skipped)
- Message stored as the parser's short message (no caret dump)
- Synthetic diagnostic code **E001**, severity Error

- Range from `SyntaxErrorDetails` (`lineno` / `offset` / `end_*`), clamped to a non-empty span on the last non-empty line for EOF errors

Quick fixes (feature module)

`create_quick_fix` is wired through `textDocument/codeAction` (`server.py` → `convert.handle_code_action`) for **W001**, **W002**, **W101**, **W104**, **W105**, and **C002**:

- **W001** (missing `//@version`) → QuickFix inserting `//@version=6\n` at document start (line 0, character 0). Offered even when Convert-to-v6 is also available (undeclared version is treated as v1 for convert).
- **W002** (`//@version=N` with N < 5) → QuickFix replacing the existing pragma **line** with `//@version=6` (does not insert a second pragma). v5 does not emit W002; Convert-to-v6 still applies.
- **W101** (v4 `security(...)` spelling) → QuickFix replacing the `security` token with `request.security`, verified by re-parsing.
- **W104** (`transp=` / `strategy.* when=` removed in v6) → QuickFix deleting the named argument and one separating comma. The AST supplies the argument and value ranges; multi-line calls are handled. The fix is not offered when the removed span contains a comment (never trade a lint fix for deleted text) or when the rewritten buffer would not parse.
- **W105** (numeric literal in a boolean position) → QuickFix rewriting the literal as `literal != 0`, preserving Pine truthiness while satisfying v6 strictness. Applied to `if`, `while`, and ternary conditions; the fix is verified by re-parsing.
- **C002** (line longer than 120 characters) → QuickFix applying the same whole-document `TextEdit` as `textDocument/formatting` (`handle_formatting` / `NodeUnparser`). Kind is `QuickFix`. Offered only when lint warnings include C002 (not invented). Parse failure, an already-canonical buffer, or a buffer containing prose/block comments (which the unparser would drop) yields no C002 fix. Convert-to-v6 / W001 / W002 are unchanged.

Capability advertises `QuickFix` and `RefactorRewrite` (Convert-to-v6). A v6 script that already has `//@version=6` has no W001/W002; C002 is still offered when the linter reports a long line and formatting would change the buffer.

Invariants and edge cases

1. **Lint does not run when parse fails** — only E001 is shown.
2. Warnings without a line number are dropped (`None` conversion).
3. Rules supply exact token columns where practical (W104/W105/W101/C001); otherwise the end column falls back to the remainder of the line and clamps to 2000 characters.
4. `inter_file_dependencies=False`: no cross-file analysis; each URI is independent.
5. Push and pull should agree for a given buffer version; both read the same `TextDocumentState`. Published diagnostics carry the document `version` so a stale debounce result cannot overwrite a newer buffer.
6. **Noise filters:** comments and string literals are masked before regex rules (W101/C001); `C001` on names that already lack `_` (camelCase `fastMA`) is dropped. The `C003` block-`if` noise filter was removed with the rule (retired 0.6.0).
7. **Style rules are opt-in in the extension.** They remain available to CLI users and other LSP clients.
8. **Semantic diagnostics are opt-in and conservative.** Only user-declared types/enums and user functions are checked; builtin arity and unknown identifiers are not guessed.
9. **C002 is not invented** — unlike W001/W002, a format QuickFix is emitted only when `warnings` contains C002, and only when formatting produces an edit.

Worked example

Source:

```
indicator("x")
plot(close)
```

If the linter emits a missing-version warning (code `W001`), the client receives a Warning diagnostic on the relevant line with `source: "pyne"`. `textDocument/codeAction` also returns a preferred QuickFix that inserts `//@version=6` at the top of the buffer.

A script with `//@version=4` gets W002 (deprecated) and a preferred QuickFix that replaces that pragma line with `//@version=6`. Convert-to-v6 remains a separate `RefactorRewrite`. v5 is not W002.

On a hard parse failure at line 3, expect:

```
{
  "severity": 1,
  "code": "E001",
  "source": "pyne",
  "message": "<exception text>"
}
```

Failure modes

Symptom	Cause
Stale squiggles after edit	Client using full-document sync incorrectly, or version mismatch
Empty diagnostics on broken file	Unexpected: parse errors should still emit E001 — check client filter on `source`
No workspace pull results	Only **open** documents are tracked; closed files are absent

See also

- Linter (core)
- Error model
- Architecture

COMPLETION

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

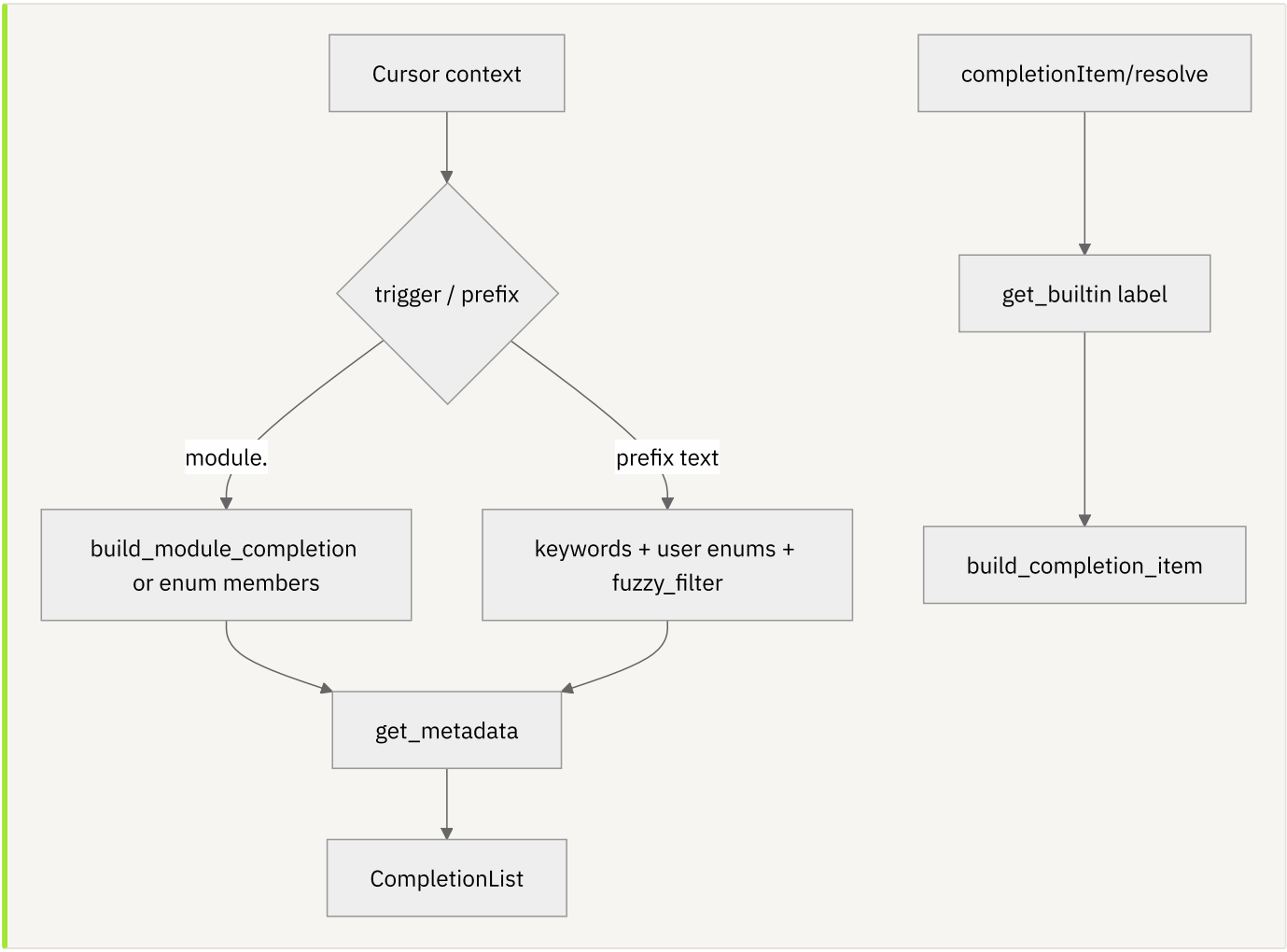
title: "Completion" description: "textDocument/completion and completionItem/resolve from builtin metadata, module-dot triggers, and fuzzy filter."

COMPLETION

Abstract

Completion combines three static sources: builtin metadata (plaintext JSON in development, Fernet-encrypted in compiled binaries), **Pine keywords**, and the **scope-aware symbol index** built from the workspace AST. The index supplies local variables, parameters, functions, methods, UDT fields, enum types/members, and imports. Member completion (`p.`) is type-directed: the receiver type is inferred conservatively and only real fields/methods are offered. Resolve re-hydrates documentation for a single builtin item.

Conceptual model



Interface surface

Method	Capability
`textDocument/completion`	`trigger_characters=["."]`, returns `CompletionList`
`completionItem/resolve`	`resolve_provider=True`

Handler: `features/completion.py` → `handle_completion` / `handle_completion_resolve`.

Trigger logic

1. Compute `text_before_cursor` on the current line.
2. Suppress completion inside comments and string literals (`in_comment_or_string`).
3. `get_trigger_char` — if the previous character is `.` (or `(`, `,`, space), treat as trigger.
4. If the last token contains `.` (e.g. `ta.`, `Side.`, `p.`, `Point.`):
   - user-enum prefix → enum members
   - UDT type prefix (`Point.`) → `new` constructor
   - receiver with an inferred UDT type (`p.`) → fields + methods
   - catalog namespace (`ta.`, `strategy.risk.`) → `build_module_completion`

5. Else merge user symbols visible in the current scope, `build_keyword_items`, enum names, builtin variables (`close`, `bar_index`, ...), and `build_completion_list(prefix=prefix)`.

Word boundaries for Pine identifiers: `[a-zA-Z_][a-zA-Z0-9_.]*` (`protocol/utils.py`).

Item fields

Built by `providers/completion_items.py`:

Field	Source
`label`	metadata `label` (e.g. `ta.sma`)
`kind`	`CompletionItemKind.Function`
`detail`	signature / detail string
`documentation`	Markdown from metadata
`insert_text`	Snippet if `${...}` present, else plain label
`insert_text_format`	Snippet or PlainText
`filter_text`	dotted parts + brief
`sort_text`	modules first (`\x01…`), root second (`\x02…`)

Category headers may appear as Folder-kind items with empty insert text and labels like `--- Technical Analysis (ta.*) (N) ---`.

Internals

Path	Role
`features/completion.py`	Request context + dispatch
`providers/completion_items.py`	List / item / module builders
`providers/builtin_metadata.py`	Load cache, `fuzzy_filter`, `get_builtin`
`protocol/utils.py`	Word + trigger helpers

Fuzzy filter scores

`fuzzy_filter(query, items, limit=50)`:

Match	Score
Exact label	1000
Prefix	500
Substring in label	100
Category	50
Brief	25

Results sorted by score descending, capped at `limit`.

Resolve

`handle_completion_resolve` looks up `params.label` in metadata; if found, rebuilds a full `CompletionItem`. Unknown labels return unchanged.

Invariants and edge cases

1. **User symbols are scope-aware.** A local `x` shadows a script-level `x`; only the nearest declaration is offered. Unknown receiver types produce no member list rather than unrelated builtins.
2. Empty metadata dict yields builtin completion without catalog items; user symbols and keywords still work.
3. Module completion uses `startswith(module + ".")` — nested namespaces beyond one dot still work if labels are fully qualified.
4. Snippets are generated at metadata build time from real handler signatures where available (`scripts/generate_builtin_metadata.py`); set `pynescript.completion.snippets` to `false` to insert plain names.
5. Mid-edit buffers that do not parse (trailing `.`, unclosed call) use a minimally repaired parse for receiver/scope resolution; positions before the cursor are unchanged.

Worked example

User types `ta.` → module completion for all `ta.*` labels.

User types `sma` without a module → fuzzy filter may return `ta.sma` among others (substring score).

Resolve on `ta.sma` reloads full documentation markup for the detail pane.

Failure modes

Symptom	Cause
Zero items always	Metadata not loaded — check builtin metadata
Dot trigger ignored	Client did not set completion trigger characters from server capabilities
Snippets inserted raw	Client disabled snippet support; extension setting `pynescript.completion.snippets`

See also

- Hover — shares metadata
- Builtin metadata
- Inlay hints — uses return types from `detail`

HOVER

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Hover" description: "textDocument/hover — Markdown cards for builtins, types, namespaces, keywords, and user declarations."

HOVER

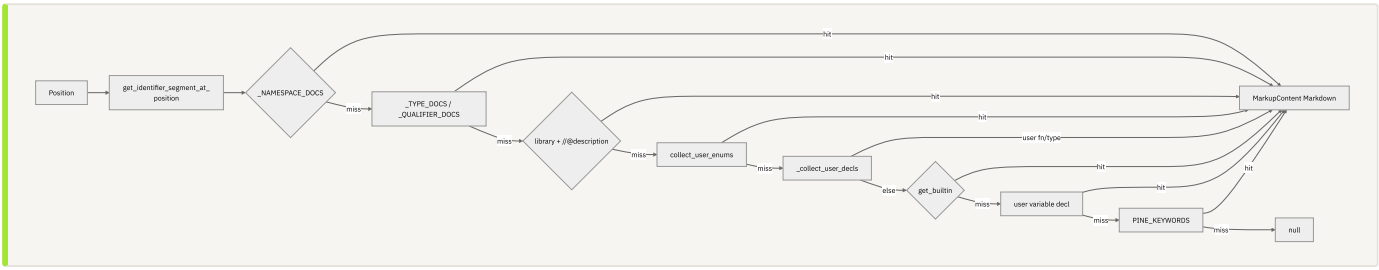
Abstract

Hover answers “what is this identifier?” using the scope-aware static index first: **local/global symbols** (variables, params, functions, methods, types, enums, enum members, fields, imports) and **type-directed members** (`p.field` , `Point.new`), then **namespace** prefix, **type / qualifier**, **builtin** metadata, built-in variables (`close` , `bar_index` , …), and **keywords**. The handler extracts the word under the cursor (including dotted forms) and

returns a Markdown card. User symbols show their inferred type and declaration signature; UDT members show their receiver. Builtin cards for dotted labels (`ta.sma` , `strategy.entry` , `line.fill`) append a Pine v6 function-reference markdown link. Keywords, types, namespaces, and user declarations stay link-free.

Library annotations become rich Markdown cards. The comment block directly above an exported declaration is parsed as Pine library annotations (`// @function` , `// @param` , `// @returns` , `// @type` , `// @field` , `// @variable` , `// @description`). Everything after the tag — including continuation `//` lines — is rendered verbatim as Markdown, so headings, bold/italic, strikethrough, blockquotes, horizontal rules, lists, code fences, tables, links, images, and two-space line breaks all appear in the hover. User-defined functions, methods, and types shadow builtin metadata, so a library function named like a builtin alias (e.g. `bb`) still shows its own docs.

Conceptual model



Interface surface

- Method: `textDocument/hover`
- Capability: `hover_provider=True`
- Handler: `features/hover.py` → `handle_hover` (receives version-matched `tree=doc.valid_ast` and optional per-version `feature_cache` for the user-decl walk)

Lookup order

1. Identifier segment at cursor via `get_identifier_segment_at_position` (identifiers may include `.`).
2. Namespace: cursor on a module name (`ta` in `ta.sma` , or bare `ta` / `math`). Types that share a name (`line` , `array` , `color`) and callables (`strategy(` , `input(`) fall through.
3. Type or qualifier (`int` , `float` , `series` , `var` , ...) when the segment is **not** followed by `(` .
4. `library` with a script-level `// @description` → library card.
5. User enum type or `Enum.member` from the cached AST.
6. User declaration lookup (`FunctionDef` / `TypeDef` / `EnumDef` / `Assign` , including tuple targets). First declaration of a name wins.
7. Builtin lookup: full word, then leaf after `.` , then `module.word` if the preceding token ends with `.` . A user **function / method / type** with the same name wins over the builtin.
8. Remaining user declarations (variables) after builtins.
9. Else keyword brief from `PINE_KEYWORDS` .
10. Build hover with range covering `[start, end)` of the word on the line.

Builtin card structure

````
```pinescript
{signature}
```

{brief}

**Parameters:** …   (when metadata includes params)

---
{first documentation paragraph, ≤500 chars}

**Example:** …
**See also:** `ta.ema`, …
[Pine v6: ta.sma](https://www.tradingview.com/pine-script-reference/v6/#fun_ta.sma)
````

Examples and related maps are **hardcoded** for a small high-value set (`ta.sma` , `ta.ema` , `ta.rsi` , `ta.macd` , strategy entry/exit, etc.). Others show signature + brief (and the docs link when the label is dotted).

`_build_docs_link` emits that markdown link only for dotted builtin labels with a namespace (`ta.sma` , `strategy.entry` , `line.fill`). Empty, undotted (`sma`), or whitespace-only names return `""` . Keyword, type, namespace, and user-declaration cards never include the TradingView URL.

Library annotation cards

Pine libraries document exported functions, methods, types, and variables with annotation comments directly above the declaration. The block is read from source (tag line **plus** every following `//` line until a non-comment line), so multi-line Markdown is preserved:

```
// @function Creates bollinger bands out of any specified moving average
//
// #### Usage
// ___
// ```
// [bb_up, bb_mid, bb_dn] = bb(close, ta.sma(close, 50), 50, 4)
// ```
// ___
// @param src   The source of the standard deviation. `Optional`
// @param mult  The multiplier of the offsets. `Required`
// @returns `Bollinger bands` of the specified moving average.

    dev = mult * ta.stdev(src, maLength)
    [movingAverage, movingAverage + dev, movingAverage - dev]
```

Hovering `bb` renders the fenced signature, the Markdown description (headings, code fence, horizontal rules), a **Parameters** list from `// @param` , and a **Returns** line. Supported tags:

Tag	Card section
`// @function` / `// @type` / `// @variable` / `// @enum`	Description (Markdown body)
`// @param <name> <desc>`	**Parameters** bullet
`// @field <name> <desc>`	**Fields** bullet
`// @returns <desc>`	**Returns** line
`// @description`	Library card (hovering `library`)

Markdown is passed through verbatim, so the TradingView Markdown in the Pine Editor formatting works: `#` headings, `**bold**` / `*italic*` / `~~strike~~`, `>` blockquotes, `___` / `***` rules, `1.` / `-` / `+` / `*` lists, ```` ``` ```` code fences, `` `code` ``, `|` tables, `[text](https://hoox.sh/pyne/docs/url)` links, `![alt](https://hoox.sh/pyne/docs/url)` images, and two trailing spaces for a line break. Both `//@function` and `// @function` spellings are accepted. Cards without a known tag fall back to `User-defined {kind}.`

Internals

Path	Role
`features/hover.py`	Word resolution + Markdown assembly
`providers/builtin_metadata.py`	`get_builtin`
`protocol/utils.py`	Identifier segment extraction

The workspace `feature_cache` memos the user-declaration walk so each hover does not re-walk the whole AST.

Invariants and edge cases

1. **No source → null.** Missing buffer aborts early.
2. **Line OOB → null.** Cursor past last line is ignored.
3. **Empty word → null.** Whitespace / punctuation yields no hover.
4. **User functions, assigns, types, and enums are hovered** — locals are not “null”. Use definition when you need a jump, not a card.
5. **Docs links are dotted builtins only.** Keywords, types, namespaces, and user decls stay link-free.
6. Documentation truncation is first-paragraph / 500-char only — full docs live in metadata and completion resolve.
7. **Annotation blocks are contiguous.** The block is the run of `//` lines immediately above the declaration. A blank line or code line ends it. Plain comments without a known `// @tag` are ignored (fallback card).
8. **User symbols shadow builtins.** A user variable/function/method/type with a builtin name wins, so a local `color` or library `bb` is described as the user declaration. Unknown identifiers fall through to builtin metadata/keywords.
9. **Markdown is never escaped or sanitized** — it is emitted as `MarkupContent(kind=Markdown)`, so clients render the author's formatting.

Worked example

Cursor on `sma` in `plot(ta.sma(close, 14))`:

- Word may be `ta.sma` if the dotted identifier is one match, or `sma` with module recovery from `ta.`.

- Response `contents.kind = markdown` with a fence of the metadata signature, plus `[Pine v6: ta.sma](https://www.tradingview.com/pine-script-reference/v6/#fun_ta.sma)`.

Cursor on `foo` in:

```
//@version=6
indicator("T")
foo(a) => a
plot(foo(1))
```

- Declaration and use both hover as a user-defined function with fence `foo(a) => a`.
- No Pine v6 URL.

Cursor on an annotated library function:

```
// @function **Bold** heading
// #### Usage
// ```
// x = foo(1)
// ```
// @param a The first.
// @returns The result.
```

- Card = fenced signature + the Markdown body (`**Bold** heading`, `#### Usage`, code fence) + `**Parameters:** - \` a`: The first.` + `**Returns:** The result.`

Failure modes

Symptom	Cause
Hover never appears	Metadata empty or word not in catalog / AST
Wrong symbol	Identifier regex swallowed neighboring text — rare with `_.` pattern
Stale docs after adding builtin	Regenerate metadata — builtin metadata
User local shows no card	Parse failed (`tree=None`) so the decl walk is skipped

See also

- Completion
- Builtin metadata
- Navigation

NAVIGATION

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Navigation" description: "Go-to-definition, find-references, document highlight, rename across open documents, document outline, and workspace symbol search over the ASDL AST."

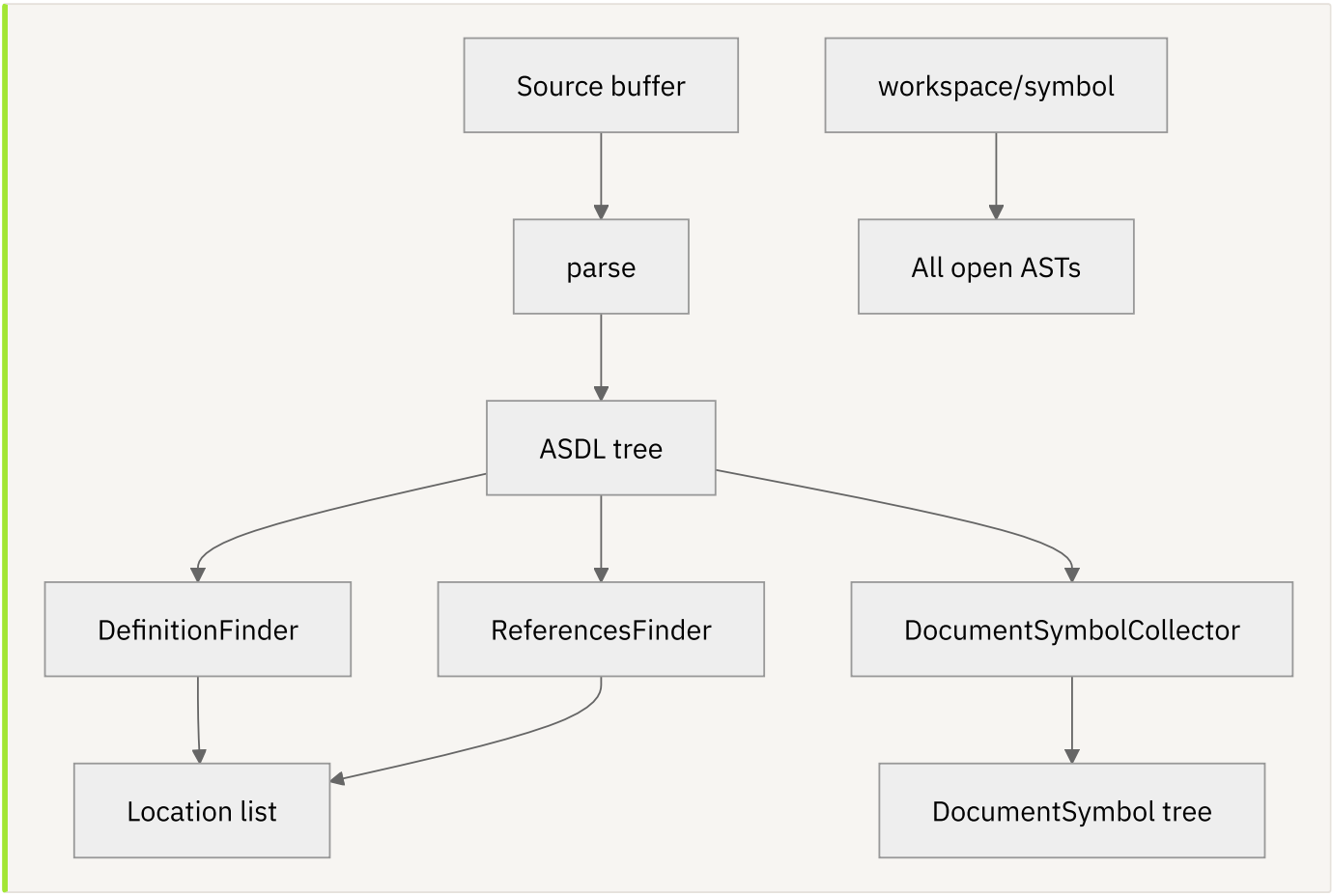
NAVIGATION

Abstract

Navigation features operate on **user AST symbols**, not the builtin catalog. A shared static index (`langserver/analysis.py`) builds Pine scopes, declarations, member references, exact identifier ranges, and conservative types. Definition/references/highlight/rename resolve the symbol under the cursor through that index, so shadowed locals and unrelated same-named fields are not conflated. Definition and reference locations are

character-precise. Rename applies to every **currently open** workspace document (script-level names also match other open scripts); document symbols build a hierarchical outline; workspace symbols include open documents plus a bounded index of closed Pine files under the workspace roots.

Conceptual model



Interface surface

Method	Handler	Returns
`textDocument/definition`	`definitions.handle_definition`	`list[Location] | None`
`textDocument/references`	`references.handle_references`	`list[Location]` (possibly empty)
`textDocument/documentHighlight`	`highlight.handle_document_highlight`	`list[DocumentHighlight]`
`textDocument/prepareRename`	`rename.handle_prepare_rename`	`Range | None`
`textDocument/rename`	`rename.handle_rename`	`WorkspaceEdit | None` (all open documents)
`textDocument/documentSymbol`	`symbols.handle_document_symbols`	`list[DocumentSymbol]`
`workspace/symbol`	`server._collect_workspace_symbols`	`list[SymbolInformation]`

Capabilities: `definition_provider`, `references_provider`, `document_highlight_provider`, `rename_provider` (`prepare_provider=True`), document + workspace symbol options with work-done progress flags.

Definition (`DefinitionFinder`)

Matches target word against:

- `FunctionDef.name`
- `TypeDef.name`
- `Assign` targets (`Name` or nested names)

Ignores mere call sites (`visit_Call` returns early when the callee name matches).

References (`ReferencesFinder`)

- `Name` with Load / Store contexts
- Function and type definitions when `include_declaration` is true
- Call sites of bare `Name` callees

`params.context.include_declaration` controls whether declarations appear.

Document symbols

Hierarchical:

AST	SymbolKind	Children
`FunctionDef`	Function	Variables assigned inside body
`TypeDef`	Class	Fields (`Assign` targets)
`EnumDef`	Enum	Members
Top-level `Assign`	Variable	—

Function `detail` is a coarse `function name()` string; assignments may show callee module attribute as detail when RHS is a call.

Workspace symbols

Flat `SymbolInformation` for functions/methods, types, enums + members, imports, and assigned names. Open buffers are authoritative; closed files under workspace roots are searched through `WorkspaceIndex` (bounded). Filter is a lowercase query substring.

Internals

Path	Role
`features/definitions.py`	Go-to-definition visitor
`features/references.py`	Find-all-references visitor
`features/highlight.py`	Document highlight + ident occurrence collector
`features/rename.py`	prepareRename (cursor document) + rename across open documents
`features/symbols.py`	Document outline
`server.py`	Workspace symbol aggregation

Handlers accept the **version-matched AST** (`tree=doc.valid_ast` from `server.py`). They re-parse from `source` only when `tree` is omitted (tests / isolated calls). Pass `tree=None` after a failed parse so the handler does not retry.

Invariants and edge cases

1. **Parse failure → empty / null** (definition `None`, others `[]`).
2. **Builtin names** have no definition in-file; clients should not expect `ta.sma` to resolve to a library file URI.
3. **Ranges are character-precise** for definition, references, highlight, rename, and document symbols. The analysis layer stores each identifier's exact `(line, column, end_line, end_column)`.
4. **Scopes are modeled** — script, function/method, if/for/while/switch/once, type, and enum. `:=` resolves outward; `=` declares locally. UDT fields/methods and enum members are separate from script variables.
5. **Closed documents** are included in workspace symbol search through the bounded `WorkspaceIndex`, but not in `textDocument/rename` or `references`; those still operate on open buffers. Rename updates every currently open buffer that contains the symbol (script-level names also match other open scripts). `prepareRename` stays the cursor document's ident range.

Worked example

```
//@version=6
indicator("nav")
length = 14
mySma(src, len) =>
    ta.sma(src, len)
v = mySma(close, length)
```

- Definition on `mySma` at the call site → function line.
- References on `length` → assign + call argument (and declaration if included).
- Outline → `length` variable, `mySma` function (with locals), `v` variable.

Failure modes

Symptom	Cause
Jump lands column 0	Expected with current Location encoding
Missing references inside function when targeting another function	`visit_FunctionDef` only walks body when the def name **differs** from target — intentional for declaration handling; calls inside same-named functions are a known edge
Empty outline	Parse error or empty source

See also

- Architecture
- Hover — builtins vs user symbols
- Diagnostics

LINKED EDITING

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

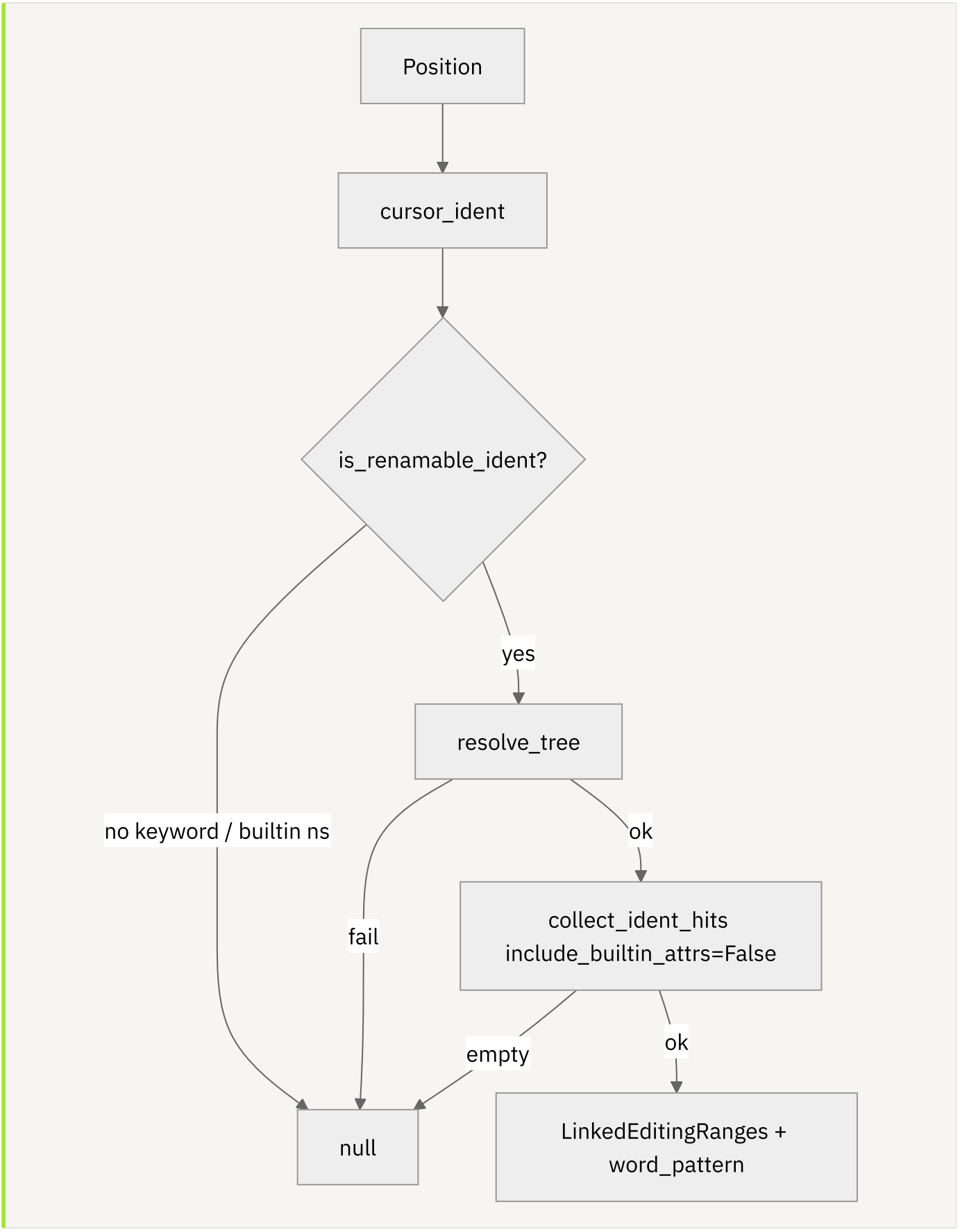
title: "Linked editing" description: "textDocument/linkedEditingRange — live multi-cursor ranges for user identifiers."

LINKED EDITING

Abstract

`textDocument/linkedEditingRange` returns same-document ranges for the **resolved user symbol** under the cursor so the client can drive live multi-cursor edits (rename-as-you-type). Resolution uses the scope-aware symbol index, so a shadowed local is not linked to a same-named global. Keywords, builtin namespaces, and unresolved identifiers stay unlinked — the same refusal set as rename. Empty buffers, unparsed documents, and non-renamable tokens return `null`.

Conceptual model



Interface surface

- Method: `textDocument/linkedEditingRange`
- Capability: `linked_editing_range_provider=True`
- Handler: `features/linked_editing.py` → `handle_linked_editing_range` (receives version-matched `tree=doc.valid_ast`)

Response:

Field	Value
`ranges`	Character-precise `Range` list for every same-document occurrence of the ident
`wordPattern`	`[A-Za-z_][A-Za-z0-9_]*`

Unlike `textDocument/rename`, linked editing is **one document** — it does not walk other open buffers.

What is unlinked

`is_renamable_ident` (shared with highlight / rename) returns false for:

- Empty or invalid tokens (not a single Pine ident)
- Keywords from `PINE_KEYWORDS` (`if`, `plot`, `enum`, ...)
- Builtin namespaces (`ta`, `strategy`, `math`, `input`, `request`, `str`, `array`, `matrix`, `map`, `color`, `line`, `label`, `box`, `table`, `polyline`, `log`, `chart`, `runtime`, `session`, `time`, `barstate`, `syminfo`, `timeframe`, `ticker`, `alert`, `hline`, ...)
- Dotted words whose head is a builtin namespace (`ta.sma` as a whole)

Hits are collected with `include_builtin_attrs=False`, so library attributes are not linked when the cursor is on a user ident.

Internals

Path	Role
`features/linked_editing.py`	LSP handler + `WORD_PATTERN`
`langserver/analysis.py`	Scope/symbol resolution and exact occurrence ranges
`features/highlight.py`	`cursor_ident`, `is_renamable_ident`
`config.py`	`linked_editing_range_provider=True`
`server.py`	`TEXT_DOCUMENT_LINKED_EDITING_RANGE`

Tests: `tests/test_lsp_features.py` (`TestLinkedEditingRangeHandler`).

Invariants and edge cases

1. **No source → null.** Missing or empty buffer aborts early.
2. **Keywords and builtin namespaces never link.** Cursor on `ta`, `plot`, or `if` yields `null`.
3. **Parse failure → null.** `tree=None` (workspace already failed) skips a redundant re-parse.
4. **Same-document only.** Cross-file live edits are rename's job (open documents), not linked editing.
5. **`wordPattern` is the Pine ident token** — the client should not extend the edit into `.` or digits-leading junk.

Worked example

```
//@version=6
indicator("t")
length = 14
plot(ta.sma(close, length))
```

Cursor on either `length`: two ranges (the assignment target and the call argument), `wordPattern` `[A-Za-z_][A-Za-z0-9_]*`. Cursor on `ta` or `plot`: `null`.

Failure modes

Symptom	Cause
Linked editing never starts	Cursor on a keyword / builtin namespace, or parse failed
Only one occurrence linked	Other hits are a different ident, or builtin attrs excluded
Edit spills into neighboring text	Client ignored `wordPattern`

See also

- Navigation — highlight and rename share the ident collector
- Architecture
- Hover — builtins vs user symbols

FORMATTING

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Formatting" description: "Document, range, and on-type formatting via AST parse → NodeUnparser round-trip."

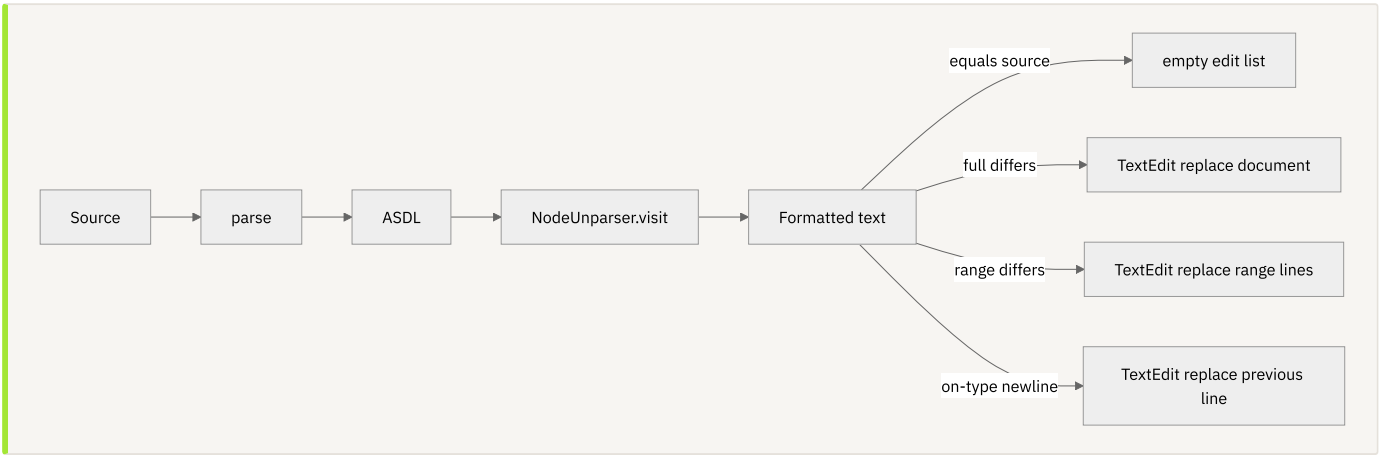
FORMATTING

Abstract

Formatting is **canonical unparse**, not a style linter. The handler parses the buffer, runs `NodeUnparser`, and if the result differs from the source, replaces the document (or the requested line range, or the just-completed line) with a single `TextEdit`. There is no prettier-like option matrix; `FormattingOptions` from the client are currently unused.

Safety rule: the ASDL AST keeps Pine annotations (`//@version` , `//@function` , …) but not ordinary prose or block comments. If the buffer contains a comment the unparser would drop, the handler switches to a **safe whitespace pass**: trailing spaces/tabs are removed and the final newline is normalized, while every comment and string is kept byte-for-byte. Lines inside multi-line strings are never trimmed. Range and on-type formatting apply the same pass to their target line. C002 does not offer a format QuickFix for such buffers because whitespace cleanup cannot shorten a long line. A full comment-preserving CST formatter is future work.

Conceptual model



Interface surface

Method	Handler
`textDocument/formatting`	`handle_formatting`
`textDocument/rangeFormatting`	`handle_range_formatting`
`textDocument/onTypeFormatting`	`handle_on_type_formatting`

Capabilities (`config.py`): both document providers `True` ; on-type advertised as `documentOnTypeFormattingProvider` with `firstTriggerCharacter = "\n"` .

Full document

1. Parse with filename `"<format>"` .
2. `formatted = NodeUnparser().visit(tree)` .
3. If identical to source → `[]` .
4. Else one `TextEdit` spanning `(0,0)` → last line/col with `new_text=formatted` .

Range

1. Unparse the **entire** document.
2. Slice **formatted** lines and **source** lines by the client range's start/end line indices.
3. If the in-range slices match → `[]` .
4. Else replace `params.range` with the formatted line slice.

This is line-aligned, not AST-node-aligned: partial mid-line ranges may produce surprising splices when unparse changes line structure.

On-type (after `\n`)

After the client inserts a newline, `params.position` is the cursor on the **new** line and `params.ch` is `"\n"`. The handler formats the **previous** line (or line 0) via the same `NodeUnparser` and, when that line differs, returns one `TextEdit` covering only that source line.

1. Empty source, `ch != "\n"`, or `tree=None` (workspace parse already failed) → `[]`.
2. Unparse the whole document.
3. `target = max(0, params.position.line - 1)`.
4. If the target is past either line list, or the formatted line equals the source line → `[]`.
5. Else replace `[target, 0) → [target, len(source_line))` with the formatted line.

Triggers other than newline are ignored. The advertised trigger is `"\n"`.

Internals

Path	Role
`features/formatting.py`	LSP handlers
`src/pynescript/ast/helper.py`	`parse`
`src/pynescript/ast/unparser.py`	`NodeUnparser`

VS Code command `pynescript.formatDocument` delegates to `editor.action.formatDocument`, which hits this provider when the language client is active. Format-on-type is the client's `onTypeFormatting` request after Enter.

Invariants and edge cases

1. **Parse failure** → `[]` (no partial format; silent no-op from the client's perspective). On-type also no-ops when the workspace already failed to parse (`tree=None`).
2. **Idempotence goal:** format twice should stabilize if unparse is canonical; any non-idempotent unparse is a core bug, not an LSP bug.
3. **Comments / trivia:** Pine annotations are preserved; prose/block comments switch to the safe whitespace pass (never a destructive edit). This is not a full CST pretty-printer.
4. **Range and on-type** require a full successful parse and matching line counts; only the edit span is clipped. If unparse changes line structure, no range/on-type edit is returned.
5. Tab size / insert spaces from `FormattingOptions` are ignored today.
6. **On-type is previous-line only** — the new empty line under the cursor is not rewritten.

Worked example

Before:

```
//@version=6
indicator("x")
plot(  close  )
```

After full format (illustrative — exact whitespace follows `NodeUnparser`):

```
//@version=6
indicator("x")
plot(close)
```

If source already matches unparse output, the server returns an empty edit list and the editor leaves the buffer alone.

On-type: typing Enter after a compact assignment `length=14` (cursor now at the start of the next line) yields one edit on that assignment line: `length = 14`.

Failure modes

Symptom	Cause
Format does nothing on broken syntax	Expected: exception → `[]`
Range format rewrites wrong lines	Unparse shifted line count; prefer full-document format
On-type after Enter does nothing	Unknown trigger, unparsed buffer, or the previous line already matches unparse
Client “format on save” loops	Unparse not idempotent — fix unparser, not client

See also

- Unparser
- Architecture
- VS Code extension

SEMANTIC TOKENS

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

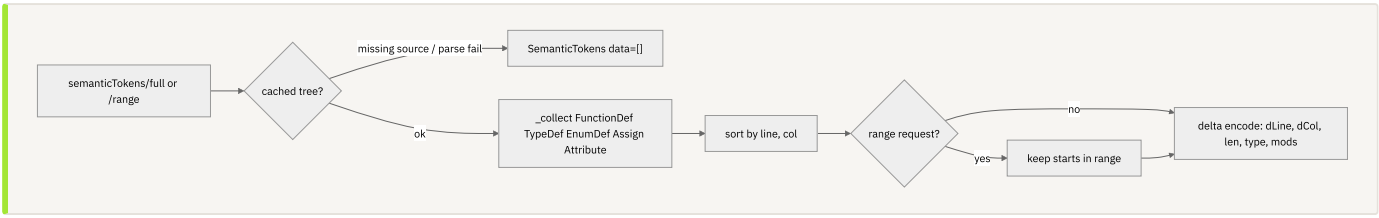
title: "Semantic Tokens" description: "textDocument/semanticTokens/full and /range — AST visitor, custom legend, and LSP delta encoding."

SEMANTIC TOKENS

Abstract

Semantic tokens let the server paint identifiers with roles richer than TextMate scopes (library namespaces vs user variables, function defs vs properties). PYNE advertises a **full and range** semantic-tokens provider with a **short custom legend** (not the full LSP standard set). `textDocument/semanticTokens/full` walks the workspace AST; `textDocument/semanticTokens/range` clips the same legend to tokens whose start falls inside the requested range.

Conceptual model



Interface surface

Capability (`config.py` → `semantic_tokens_provider`):

```
legend.token_types (index order is contract):
  namespace, type, class, function, method, variable,
  parameter, property, keyword, string, number, operator, comment

legend.token_modifiers (bit 0 = declaration):
  declaration, definition, readonly, defaultLibrary

range: true
full: true
```

Method	Handler
`textDocument/semanticTokens/full`	`handle_semantic_tokens`
`textDocument/semanticTokens/range`	`handle_semantic_tokens_range`

Both take `(params, source, tree=…)` and return `lsp.SemanticTokens(data=[...])` — LSP-encoded five-tuples as a flat `int` array (relative line/character deltas). Range uses the **same legend** as full: collect, sort, then keep tokens whose start `(line, col)` is in `params.range` (inclusive start, exclusive end) before encoding.

What is emitted

Symbol occurrence	Token type	Modifiers
Function / method declaration	function / method	definition | declaration
Type declaration	class	definition | declaration
Enum declaration	type	definition | declaration
Enum member / UDT field	property	declaration on the member
Variable declaration and reads	variable	declaration on the binding
Parameter declaration and reads	parameter	declaration on the binding
Import alias and uses	namespace	declaration on the alias
Builtin namespace attribute (`ta.sma`)	namespace + method	defaultLibrary (+ readonly on ns)
Other attribute (unknown receiver)	property	—

Builtin namespace set (`_BUILTIN_NS`): `ta` , `math` , `str` , `array` , `matrix` , `map` , `strategy` , `request` , `input` , `color` , `line` , `label` , `box` , `table` , `polyline` , `log` , `ticker` , `timeframe` , `chart` , `runtime` , `syminfo` , `barstate` , `session` , `time` .

Internals

Path	Role
`features/semantic_tokens.py`	Collector + `_encode` + range clip
`config.py`	Legend + capability — **indices must match** `_TT`
`server.py`	`TEXT_DOCUMENT_SEMANTIC_TOKENS_FULL` / `_RANGE`; passes version-matched `doc.valid_ast`

Attribute columns prefer `end_col_offset - len(attr)` when present; otherwise parent `col_offset` (approximate).

TextMate from `vscode-extension/syntaxes/pinescript.tmLanguage.json` still colors keywords, strings, and comments — the visitor does not emit those yet even though the legend reserves the slots.

Invariants and edge cases

1. **Never throws** — missing source, `tree=None`, or parse exception yields empty `data`.
2. **Range is a clip of full** — same types/modifiers; tokens whose start is outside the range are dropped. A range token stream is a subsequence of the full stream.
3. **Legend is fixed** at initialize; changing indices later would break cached clients.
4. Empty `data` is valid protocol (empty file, parse failure, or no tokens in range), not an error.
5. Tokens are sorted by `(line, col)` before delta encoding.

Worked example

```
//@version=6
indicator("tok")
length = 14
plot(ta.sma(close, length))
```

Full document: tokens for `length` (variable/declaration), `ta` (namespace/defaultLibrary), `sma` (method/defaultLibrary). Keywords such as `indicator` remain TextMate-only.

Range covering only the `plot(ta.sma…)` line keeps `ta` / `sma` (and any other starts on that line) and drops the `length` assignment. A range covering only the `//@version=6` pragma yields no namespace token.

Failure modes

Symptom	Cause
Semantic highlighting off despite tokens	Client `editor.semanticHighlighting.enabled` false
Colors look “shifted” after a legend edit	Client cached old type indices — restart the language client
Range request returns empty `data`	Parse failure, empty buffer, or no token starts in range — not a capability miss
Attribute token sits on the wrong column	Missing `end_col_offset` on the AST node — approximate path

See also

- VS Code extension — TextMate grammar
- Architecture
- Inlay hints — complementary visual annotations

INLAY HINTS

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

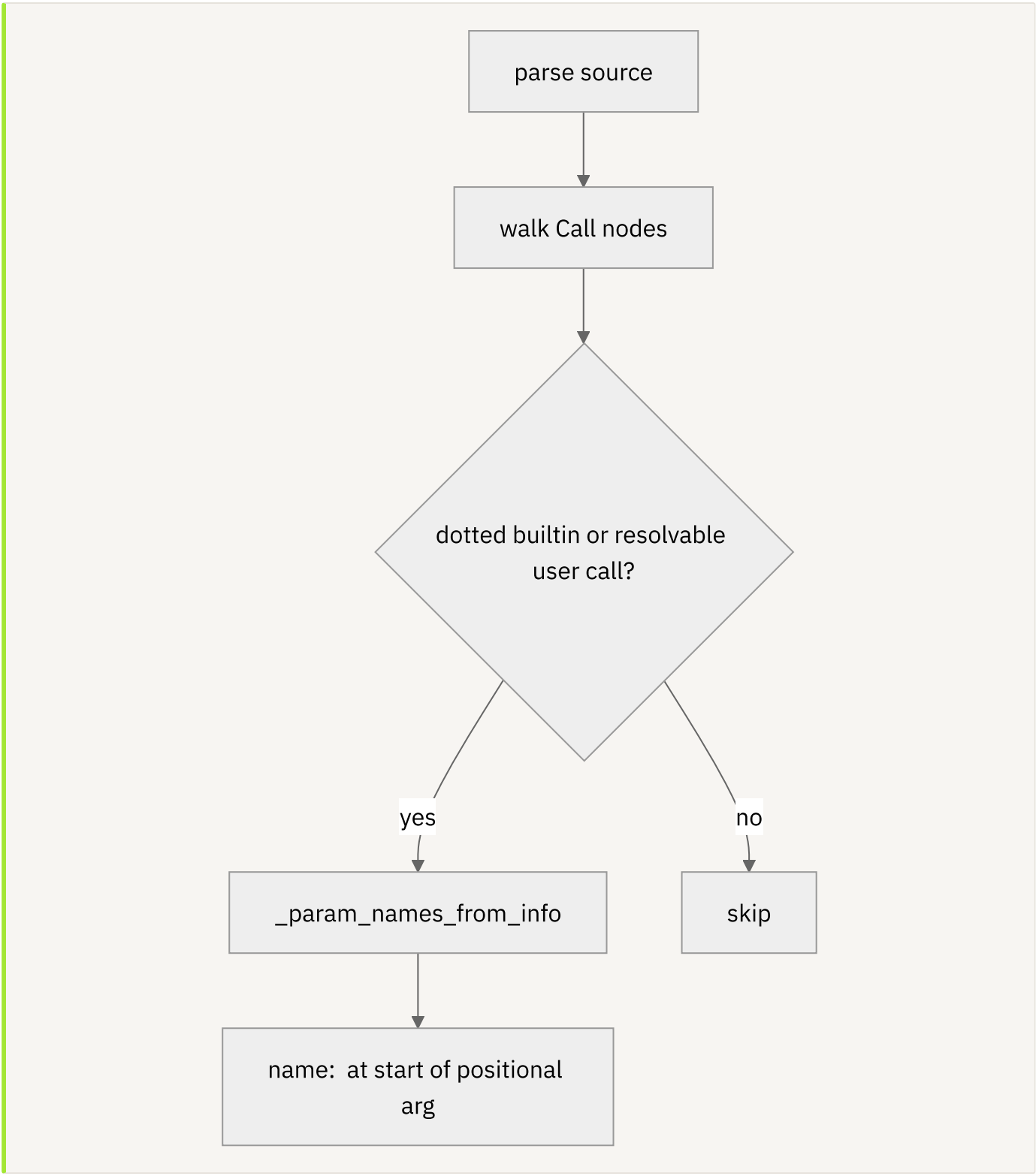
title: "Inlay Hints" description: "Display-only builtin parameter names. Hints never insert text or modify the document."

INLAY HINTS

Abstract

Inlay hints surface **parameter names** before positional arguments of builtin and user function/method calls (`ta.sma(close, 14)` → `source:` / `length:`). Only confident cases are emitted: ambiguous receivers are skipped rather than guessed wrong. Hints are filtered to the client's requested range. **Hints are display-only:** the server does not advertise or implement `inlayHint/resolve`, emits no `data`, and never sets `textEdits`, so a hint can never be injected into the source. Inferred assignment type hints were removed entirely — the server never annotates declarations.

Conceptual model



Example mental model (from module docstring):

```
ta.sma(close, 14)          →  ta.sma(source: close, length: 14)
```

Interface surface

Method	Capability
`textDocument/inlayHint`	`InlayHintOptions(resolve_provider=False)`

Handler: `features/inlay_hints.py` → `handle_inlay_hints` (receives the version-matched `tree=doc.valid_ast`). There is no `inlayHint/resolve` handler and no document mutation path.

Parameter-name hints

Emitted for positional args of `module.attr(...)` when `get_builtin("module.attr")` hits. Real snippet placeholder names win; generic `paramN` snippets fall through to the longest of the `detail` signature, documentation, and a well-known Pine v6 map (`ta.sma` / `ema` / `rsi` → `source` / `length`, `ta.atr` → `length`, `ta.correlation` → `source1` / `source2` / `length`, `strategy.entry` → `id` / `direction` / `qty`, `input.int` → `defval` / `title`, plus `ta.cci` / `roc` / `mom` / `linreg` / `adx` / `mfi` / `bbw` / `dpo` / `crossover` / `crossunder`).

Field	Value
`position`	Start of the positional argument
`label`	`"{name}: "`
`kind`	`InlayHintKind.Parameter`
`data`	omitted
`textEdits`	omitted (display-only)

Named kwargs (`length=14`) skip the matching slot.

`handle_inlay_hints` returns `[]` for empty source, `None` if parse fails (client typically treats as no hints).

Internals

Walk

Recursive over `_fields` (same pattern as workspace symbol collection) — not limited to script top-level; nested calls can receive hints.

Path	Role
`features/inlay_hints.py`	Collection (display-only)
`providers/builtin_metadata.py`	Parameter names

Invariants and edge cases

1. **Metadata quality bounds accuracy** — missing param names yields no parameter hints.
2. **Hints never mutate the document.** No `textEdits`, no `data`, no resolve handler; applying a hint is a client-side display action only.
3. **No inferred type hints.** The server never annotates declarations — the `show_types` path and its inference helpers were removed.
4. **Malformed input does not throw** — unparsable or empty sources return `None` / `[]`.

Worked example

```
//@version=6
indicator("hints")
length = 14
src = close
avg = ta.sma(src, length)
period = input.int(20, "Period")
```

Expected hints (when parse + metadata succeed):

- `length` → `: const int` (type)
- `src` → `: series float`
- `avg` → type from `ta.sma` detail (typically series float) or `series`
- `period` → `: input int`
- `ta.sma(src, length)` → `source:` before `src`, `length:` before `length`

`avg` will not get a hint derived from `length`'s type through the call graph — only the call's known return shape. Clicking a hint never edits the call or the declaration.

Failure modes

Symptom	Cause
No hints on file	Parse error → `None`
`ta.*` always `: series`	Metadata detail lacks `→` segment
No `source:` / `length:` on a builtin call	Not a dotted builtin, named kwargs, or metadata/snippet names missing
Applying a hint does nothing	Expected: hints are display-only; there is no type injection
Hints mid-identifier	Incorrect `col_offset` on AST node — builder issue

See also

- Completion
- Builtin metadata
- Type system

DOCUMENT LINK

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

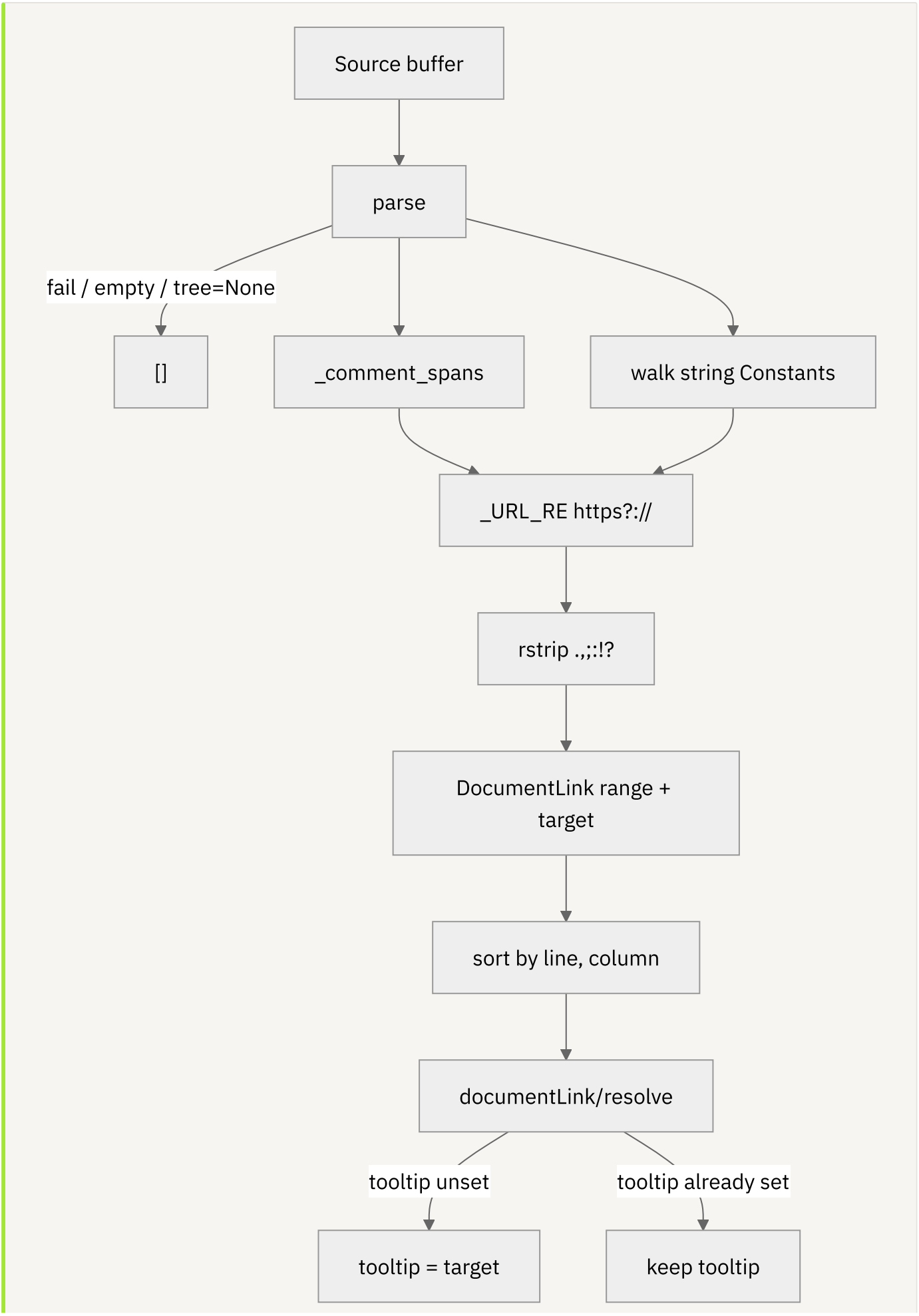
title: "Document links" description: "textDocument/documentLink for http(s) URLs in comments and strings, and documentLink/resolve tooltip."

DOCUMENT LINKS

Abstract

`textDocument/documentLink` reports `http://` / `https://` URLs found in comments (`//` , `/* */` , hash `# …`) and string literals. Each match is a `DocumentLink` whose range is the URL (not the surrounding quotes or comment markers) and whose `target` is that URL. Empty source or parse failure returns `[]` . The first response leaves `tooltip` unset; `documentLink/resolve` copies the URL onto `tooltip` when it is unset so clients can show it on hover. Advertised as `DocumentLinkOptions(resolve_provider=True)` .

Conceptual model


Source buffer
parse
fail / empty / tree=None
[]
_comment_spans
walk string Constants
_URL_RE https?://
rstrip .,;:!?
DocumentLink range + target
sort by line, column
documentLink/resolve
tooltip unset
tooltip already set
tooltip = target
keep tooltip

Comments are scanned from the raw buffer so they do not need AST nodes. String URLs come from `Constant` nodes whose `value` is a `str`. Hash comments are distinguished from `#rgb` color tokens: `#` followed by a hex digit is a color, not a comment.

Interface surface

Method	Capability
`textDocument/documentLink`	`DocumentLinkOptions(resolve_provider=True)`
`documentLink/resolve`	fills `tooltip` with the URL when tooltip is unset

Handlers: `features/document_link.py` → `handle_document_link` (receives version-matched `tree=doc.valid_ast`) and `handle_document_link_resolve`.

Each initial result:

Field	Value
`range`	LSP range covering the URL text
`target`	The URL string (`http://` or `https://`)
`tooltip`	unset (`None`) until resolve

Returns `[]` for empty source, parse failure, or `tree=None`.

Where URLs are collected

Region	Source
`//` line comments	Raw buffer, through the newline (or EOF)
`/* */` block comments	Raw buffer, including the closers; unclosed blocks run to EOF
Hash `# …` comments	Raw buffer, when `#` is not a `#rgb` color token
String literals	AST `Constant` with a `str` value; range is the URL inside the quotes

The matcher is `http://` or `https://` followed by non-whitespace characters excluding `<>"'` and `*`. Trailing `.,;:!?` is stripped from both the range and the `target` (`// see https://example.com/end.` → `https://example.com/end`).

Not collected:

- `ftp://…`, scheme-less `www.example.com`, or other non-`http(s)` schemes
- URLs in identifier / number tokens (only comments and string Constants)
- `#hex` color literals (they are not hash comments)

Resolve

`documentLink/resolve` fills `tooltip` with the link `target` URL:

1. An existing tooltip is kept (already-resolved links are returned unchanged).
2. When tooltip is unset and `target` is a non-empty string, `tooltip` is set to that URL.
3. Empty or malformed links are returned unchanged (never raises).

Initial `textDocument/documentLink` responses already carry `target`. Resolve copies that URL onto `tooltip` so clients can show it on hover.

Internals

Path	Role
`src/pynescript/langserver/features/document_link.py`	Comment scan + string walk + resolve
`src/pynescript/langserver/config.py`	`DocumentLinkOptions(resolve_provider=True)`
`src/pynescript/langserver/server.py`	`TEXT_DOCUMENT_DOCUMENT_LINK` (cached on `doc.feature_cache["document_link"]`); `DOCUMENT_LINK_RESOLVE`

The workspace cache supplies `tree=doc.valid_ast`. Pass `tree=None` when the last parse failed (skips a redundant re-parse). Omit `tree` only in isolated tests (handler then parses from `source`). Comments need the source text; a pre-parsed tree with `source=None` still yields string-literal links.

Tests: `tests/test_lsp_features.py` (`TestDocumentLinkHandler`).

Invariants and edge cases

1. **Empty / unparsed → `[]`.** Missing source, parse exception, or `tree=None` never returns `None`.
2. **Range is the URL, not the quotes.** `"https://example.com"` highlights `https://example.com`.
3. **Trailing punctuation is not part of the link.** `https://example.com/end.` strips the period.
4. **Hash comments vs colors.** `# hash https://…` is a comment; `#ff0000` is a color token and is not scanned as a comment.
5. **Results are sorted** by start line, then start column.
6. **Workspace cache.** The server stores the `textDocument/documentLink` result on `feature_cache` until the buffer changes. Resolve is not cached; it fills `tooltip` on the link the client sends back.
7. **Resolve fills tooltip only when unset.** An existing tooltip is kept; empty or malformed links are returned as-is and never throw.

Worked example

```
//@version=6
indicator("T")
// docs: https://example.com/pine
url = "https://tradingview.com/pine-script-docs"
also = 'http://example.org/path'
/* block https://example.com/block and https://example.com/two */
# hash https://example.com/hash
// trailing https://example.com/end.
plot(close)
```

Expected links (`target` equals the range text; `tooltip` is unset until resolve):

- `https://example.com/pine`
- `https://tradingview.com/pine-script-docs`

- `http://example.org/path`
- `https://example.com/block`
- `https://example.com/two`
- `https://example.com/hash`
- `https://example.com/end` (period not included)

After `documentLink/resolve`, each of those links has `tooltip` equal to `target` (for example `https://example.com/pine`). A client that already set a tooltip keeps that string.

`ftp://example.com` and `www.example.com` in a comment produce no links.

Failure modes

Symptom	Cause
No clickable URLs	Parse failed (`[]`), or the URL is not `http(s)`
Link includes a trailing period	Client ignored the range and used line text; the server strips `.,;:!?`
Comment URL missing, string URL present	Handler ran with `source=None` and a cached tree — comments need the buffer
`#ff0000` not a link	Expected: hash color tokens are not comments
Hover over a link shows no tooltip	Client skipped `documentLink/resolve`, or the link had no `target`
Tooltip overwritten	Not expected: resolve keeps an existing tooltip

See also

- Document color — `#hex` vs hash comments
- Architecture — declared capabilities and AST cache
- Hover — Pine v6 function-reference links on builtin cards

DOCUMENT COLOR

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

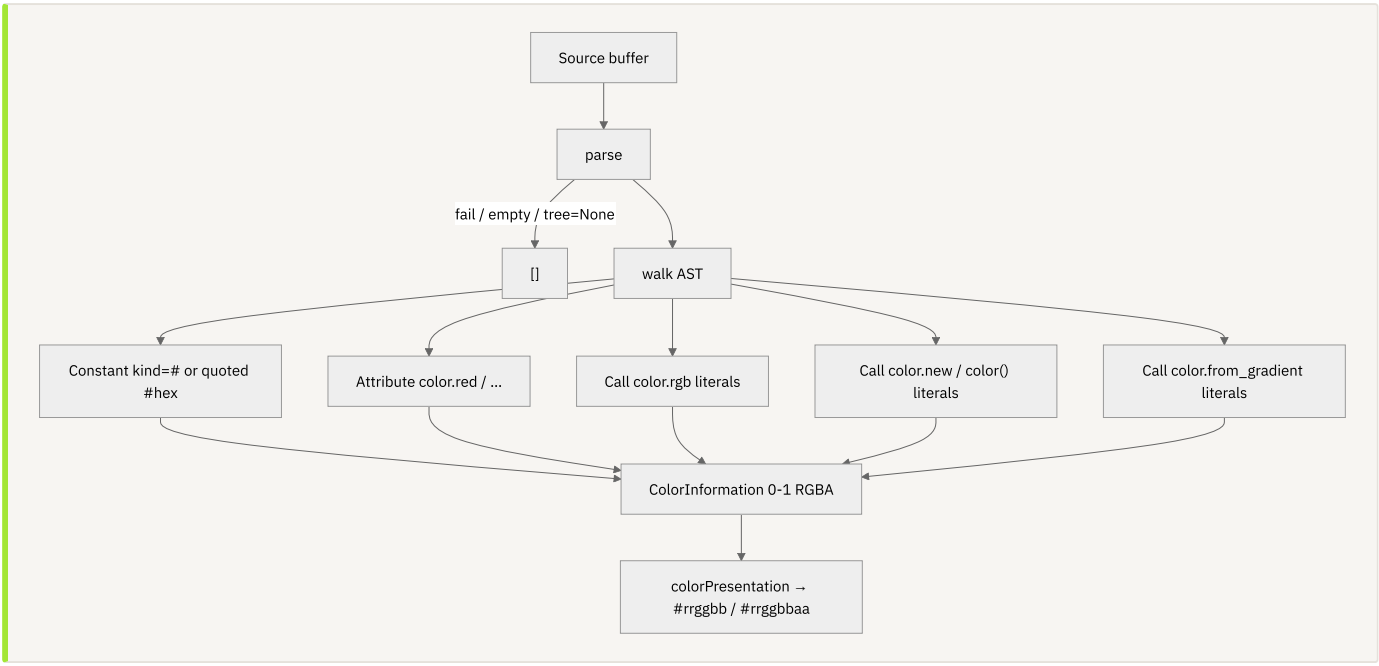
title: "Document color" description: "textDocument/documentColor and colorPresentation for #hex, color.*, and literal color calls."

DOCUMENT COLOR

Abstract

`textDocument/documentColor` reports editor color swatches for Pine color literals. Each hit is a 0–1 RGBA `ColorInformation` covering the AST node. `textDocument/colorPresentation` rewrites the requested range as `#rrggbb` (or `#rrggbbaa` when alpha ≠ 1), including when the original text was a `color.*` name. Empty source or parse failure returns `[]`.

Conceptual model



Only **literal** values become swatches. A `color.rgb(x, 0, 0)` whose `x` is a name is skipped; a nested `color.blue` inside a skipped call still gets its own swatch.

Interface surface

Method	Handler
`textDocument/documentColor`	`handle_document_color`
`textDocument/colorPresentation`	`handle_color_presentation`

Capability: `color_provider=True` (`config.py`). `handle_document_color` receives the version-matched AST (`tree=doc.valid_ast`).

Returns `[]` for empty source, parse failure, or `tree=None`. Never `None`.

Reported swatches

Form	Example	Notes
Lexer `COLOR` token	`#ff0000`, `#00ff00aa`, `#f0a`	`Constant` with `kind="#"`; `#rgb` expands each nibble (`#f0a` → `ff00aaff`)
Quoted hex string	`"#ff0000"`, `"#00ff00aa"`	`Constant` with `kind is None`; range **includes the quotes**
Named `color.*`	`color.red`, `color.green`	`Attribute` on `Name("color")`; palette from `_NAMED_COLORS`
`color.rgb`	`color.rgb(255, 0, 0[, transp])`	Numeric literals only; optional Pine transp 0–100 (0 = opaque)
`color.new`	`color.new(color.white, 50)`	Both args literals (named / `#hex` / quoted hex + numeric transp)
`color()`	`color(#ff0000, 25)`	Pine alias of `color.new`
`color.from_gradient`	`color.from_gradient(5, 0, 10, #000000, #ffffff)`	All five args literals; interpolation matches interpret `color.from_gradient`

`#rrggbb` is opaque (alpha 1). `#rrggbbaa` sets alpha from the last byte. Channels are 0–255, divided by 255 for the LSP `Color`.

Pine transparency

`color.rgb` / `color.new` / `color()` take **transp 0–100**, where **0 is opaque**. Alpha byte = `round(255 * (1 - transp / 100))`. Example: transp `50` → alpha `128`; transp `95` → alpha `13`.

Keyword aliases on `color.rgb`: `r` / `g` / `b` / `t` for `red` / `green` / `blue` / `transp`. `from_gradient` accepts evaluator aliases (`min` / `max` / `color1` / `color2`, …).

colorPresentation

Rewrites `params.range` as hex. `source` is unused — the client already sent the color and range.

Alpha	Label / `newText`
1 (`255`)	`#rrggbb` (lowercase)
otherwise	`#rrggbbaa`

Picking a color on `color.red` therefore replaces the name with `#f23645`. There is no presentation that keeps the `color.*` spelling.

Internals

Path	Role
`src/pynescript/langserver/features/document_color.py`	Walk + RGBA conversion + presentation
`src/pynescript/ast/evaluator/builtins/color.py`	`_NAMED_COLORS`, `color_from_gradient`
`src/pynescript/langserver/config.py`	`color_provider=True`
`src/pynescript/langserver/server.py`	`TEXT_DOCUMENT_DOCUMENT_COLOR` / `TEXT_DOCUMENT_COLOR_PRESENTATION`; color list cached on `doc.feature_cache["document_color"]`

Named palette (hex as stored): `red` `#F23645`, `green` `#22AB94`, `blue` `#2962FF`, `black`, `white`, `gray` / `grey`, `orange`, `purple`, `yellow`, `aqua`, `fuchsia`, `lime`, `maroon`, `navy`, `olive`, `silver`, `teal`.

`from_gradient` lerp matches interpret: RGBA interpolate, clamp `value` to `[min, max]`, equal min/max → color1.

Tests: `tests/test_lsp_features.py` (`TestDocumentColorHandler`).

Invariants and edge cases

1. **Empty / unparsed → `[]`.** Missing source, parse exception, or `tree=None`.
2. **Non-literal args skip the call**, not nested literals. `color.rgb(x, 0, 0)` is not a swatch; `color.blue` inside a skipped `color.rgb(color.r(color.blue), …)` still is.
3. **Unknown `color.foo` / `color.r` are not swatches.** Only names in `_NAMED_COLORS`.
4. **`color.new` / `color()` need both args.** One-arg `color.new(color.white)` is skipped. Extra args skip the call. Nested `color.rgb(...)` as the color arg is not a literal color.
5. **Quoted non-hex strings are ignored.** `"not-a-color"` and `"ff0000"` (no `#`) are not swatches.
6. **Presentation always hex.** Named ranges become `#rrggbb` / `#rrggbbaa`; the handler does not emit `color.red` as a presentation label.
7. **Hash comments vs colors.** `#ff0000` is a color token; `# hash …` is a comment (see document links).

Worked example

```
//@version=6
indicator("T")
c1 = #ff0000
c2 = #00ff00aa
plot(close, color=#2962FF)
bgcolor(#22AB94)
plot(close, title="#ff00ff")
col = color.red
c = color.rgb(255, 0, 0)
d = color.rgb(33, 150, 243, 95)
n = color.new(color.white, 50)
g = color.from_gradient(5, 0, 10, #000000, #ffffff)
```

Expected swatches (0–1 RGBA, 8-bit shown):

Text	RGB (alpha)
`#ff0000`	255, 0, 0 (255)
`#00ff00aa`	0, 255, 0 (170)
`#2962FF`	41, 98, 255
`#22AB94`	34, 171, 148
`"#ff00ff"`	255, 0, 255 (range includes quotes)
`color.red`	242, 54, 69 (`#F23645`)
`color.rgb(255, 0, 0)`	255, 0, 0
`color.rgb(33, 150, 243, 95)`	33, 150, 243 (13)
`color.new(color.white, 50)`	255, 255, 255 (128)
`color.from_gradient(5, 0, 10, #000000, #ffffff)`	127, 127, 127 (midpoint)

`colorPresentation` on `#ff0000` with opaque red yields label `#ff0000`. On `color.red` with the palette red, the edit replaces the name with `#f23645`.

Not swatches: `color.rgb(x, 0, 0)`, `color.from_gradient(close, 0, 100, color.red, color.green)`, `color.new(color.white)` (missing transp).

Failure modes

Symptom	Cause
No swatches on file	Parse error → `[]`
`color.rgb(close, 0, 0)` has no picker	Non-literal args — expected skip
Picking a color on `color.red` rewrites hex	Expected: presentation is always `#rrggbb` / `#rrggbbaa`
Quoted `"#ff0000"` picker covers the quotes	Expected: range is the whole string Constant
Gradient swatch disagrees with chart	Call args were not all literals, or min/max equal (then color1)

See also

- Document links — hash comments vs `#hex` tokens
- Inlay hints — parameter-name labels in calls
- Architecture — `color_provider=True`

FOLDING

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

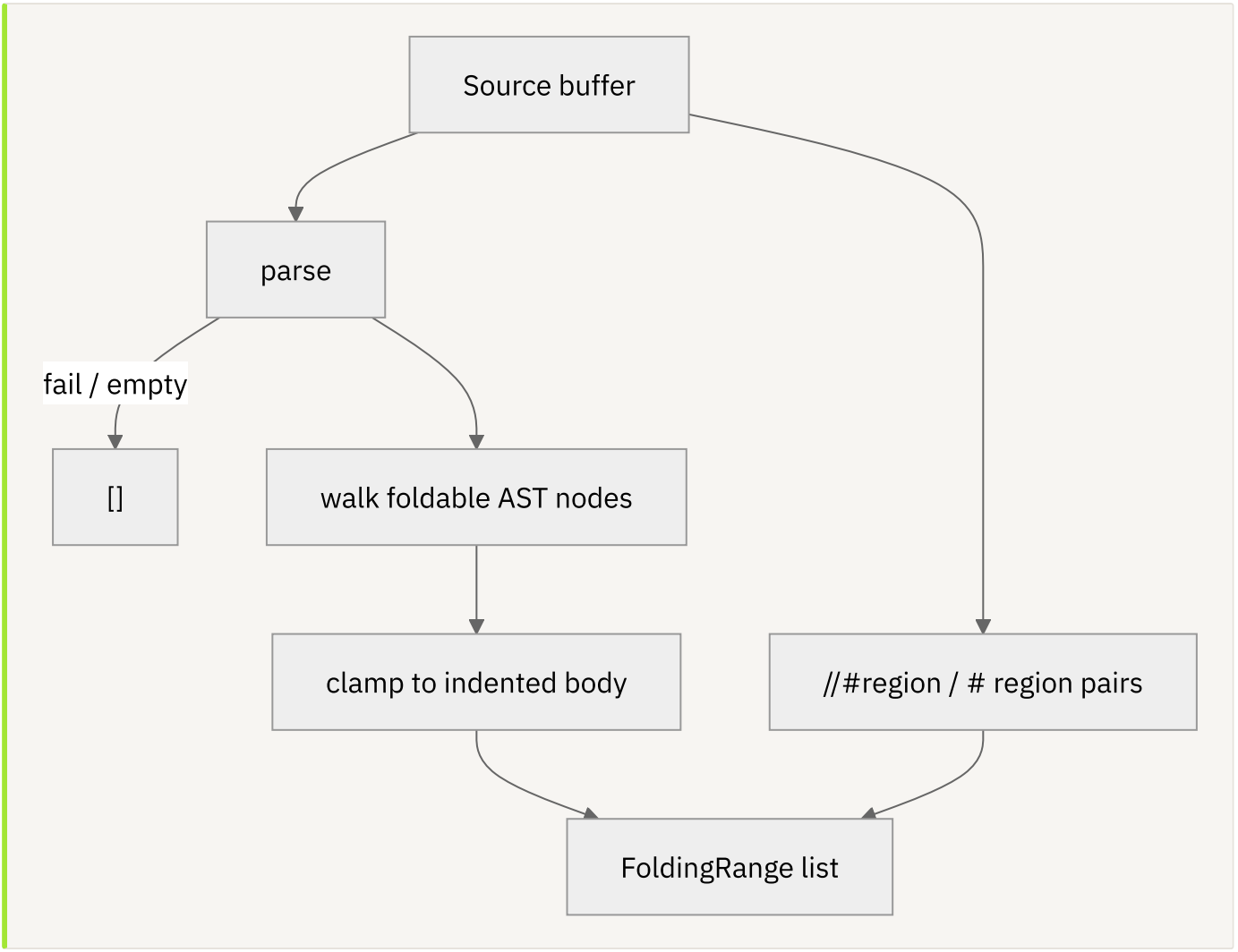
title: "Folding ranges" description: "textDocument/foldingRange for Pine blocks, type/enum/function bodies, and //#region comments."

FOLDING RANGES

Abstract

`textDocument/foldingRange` folds indented Pine blocks so editors can collapse `if` / `for` / `while` / `switch` / `once`, `FunctionDef` / `TypeDef` / `EnumDef`, and `//#region` … `//#endregion` pairs. Empty or unparsed documents return `[]`.

Conceptual model



Control blocks are expressions (`If` , `ForTo` , `ForIn` , `While` , `Switch` , `Once`) and often sit under `Expr` . The walker visits the inner node. Parser `end_lineno` can land on the next statement, so the handler prefers the indented body when source lines are available (`else` at the header indent continues an `if`). Single-line constructs are skipped.

Interface surface

- Method: `textDocument/foldingRange`
- Capability: `folding_range_provider=True`
- Handler: `features/folding.py` → `handle_folding_range` (receives version-matched `tree=doc.valid_ast`)

Each range uses 0-based `start_line` / `end_line` . Region comment pairs set `kind: "region"` .

Construct	Folded
`if` / `else if` / `else`	keyword line through the last indented body line
`for` … `to` / `for` … `in`	same
`while` / `switch` / `once`	same
`FunctionDef` / `TypeDef` / `EnumDef`	declaration through last body line
`//#region` … `//#endregion`	start through end marker (`# region` / `# endregion` too)

Returns `[]` for empty source, parse failure, or `tree=None`.

Internals

Path	Role
`src/pynescript/langserver/features/folding.py`	AST walk + indent clamp + region scan
`src/pynescript/langserver/config.py`	`folding_range_provider=True`
`src/pynescript/langserver/server.py`	`TEXT_DOCUMENT_FOLDING_RANGE` handler

Tests: `tests/test_lsp_features.py`.

See also

- Architecture — declared capabilities
- Selection ranges — expand selection over the same blocks
- Navigation — outline symbols share AST line spans

SELECTION RANGE

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

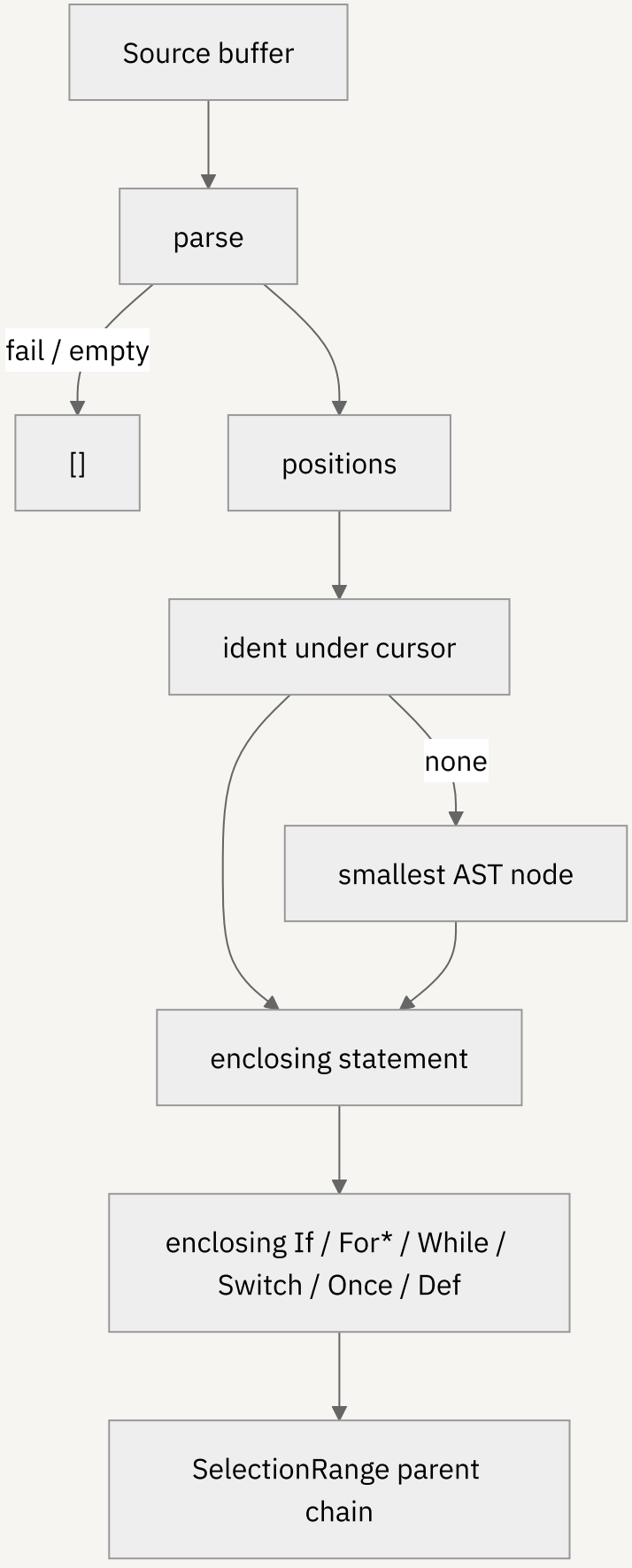
title: "Selection ranges" description: "textDocument/selectionRange expands from an ident or smallest AST node to the enclosing statement and block."

SELECTION RANGES

Abstract

`textDocument/selectionRange` grows the editor selection from the identifier (or smallest AST node) under the cursor to the enclosing statement, then enclosing `if` / `for` / `while` / `switch` / `once` blocks and `FunctionDef` / `TypeDef` / `EnumDef`. Empty or unparsed documents, or a position with no node, return `[]`.

Conceptual model



Control blocks are expressions (`If` , `ForTo` , `ForIn` , `While` , `Switch` , `Once`) and often sit under `Expr` . Duplicate spans (the `Expr` wrapper and the inner block) are collapsed. Nested blocks are linked inner-to-outer so Expand Selection walks ident → statement → inner block → outer block.

Interface surface

- Method: `textDocument/selectionRange`
- Capability: `selection_range_provider=True`
- Handler: `features/selection_range.py` → `handle_selection_range` (receives version-matched `tree=doc.valid_ast`)

Each result is a `SelectionRange` whose `range` contains the requested position and whose optional `parent` is a strictly larger containing range (LSP exclusive-end ranges from AST `lineno` / `end_lineno` / `col_offset` / `end_col_offset`).

Layer	Selected
Innermost	Identifier segment under the cursor, else the smallest AST node
Statement	Enclosing `stmt` (`Assign` , `Expr` , `FunctionDef` , …)
Block	`If` / `ForTo` / `ForIn` / `While` / `Switch` / `Once` / `FunctionDef` / `TypeDef` / `EnumDef`

Returns `[]` for empty source, parse failure, `tree=None` , or no node at the position.

Internals

Path	Role
`src/pynescript/langserver/features/selection_range.py`	Ident + AST walk → nested `SelectionRange`
`src/pynescript/langserver/config.py`	`selection_range_provider=True`
`src/pynescript/langserver/server.py`	`TEXT_DOCUMENT_SELECTION_RANGE` handler

Tests: `tests/test_lsp_features.py` .

See also

- Architecture — declared capabilities
- Folding ranges — same block constructs, line folds
- Navigation — outline symbols share AST line spans

BUILTIN METADATA

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Builtin Metadata" description: "JSON catalog for LSP completion and hover; generation pipeline; Fernet encryption, CRYPTO_KEY / METADATA_KEY, and decrypt path."

BUILTIN METADATA

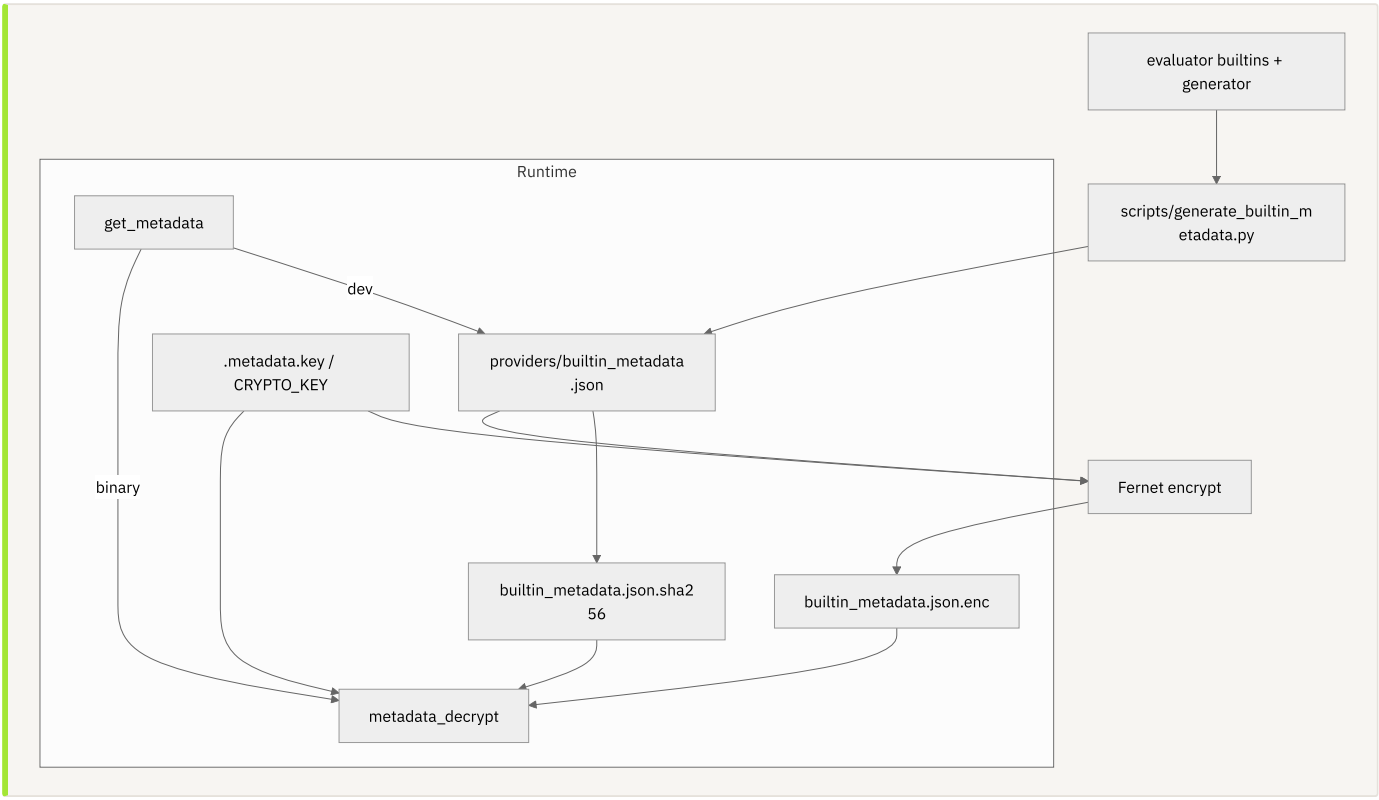
Abstract

Builtin metadata is the **LSP documentation corpus**: a map from fully qualified names (`ta.sma`, `strategy.entry`, ...) to labels, signatures, briefs, snippets, parameter names, and categories. It is generated from code (not hand-edited as source of truth): handler docstrings, `inspect.signature`, `_KWARG_ORDER`, and `_TA_KWARG_ORDERS` supply

real parameter names where available; unknown arity falls back to the bare name rather than a bogus `name(arg)` snippet. The JSON is loaded by the language server for completion/hover/signature help/inlay and optionally **Fernet-encrypted** for Nuitka onefile distribution so casual string extraction does not dump the full catalog.

This is obscurity for distribution hygiene, not a cryptographic access-control boundary.

Conceptual model



Interface surface (metadata record)

Typical entry in `builtin_metadata.json`:

```
{
  "ta.sma": {
    "label": "ta.sma",
    "kind": "function",
    "detail": "ta.sma(source, length)",
    "brief": "Simple Moving Average.",
    "documentation": "…",
    "snippet": "ta.sma(${1:source}, ${2:length})",
    "params": ["source", "length"],
    "category": "ta.technical_analysis"
  }
}
```

Categories drive completion headers (`ta.technical_analysis` → “Technical Analysis (ta.*)”, `builtin` → “Built-in Variables”, etc.).

Loader API

`providers/builtin_metadata.py`:

Function	Behavior
`get_metadata()`	Singleton cache; **plaintext JSON first**, else `.enc` decrypt, else `{}`
`get_builtin(name)`	Single entry or `None`
`get_builtins_by_category` / `get_all_categories`	Category queries
`fuzzy_filter`	Scored substring match for completion

Consumers

- Completion list / module / resolve
- Hover cards
- Inlay return-type extraction from `detail` after `→`

Internals

Path	Role
`scripts/generate_builtin_metadata.py`	Introspect builtins → JSON
`src/pynescript/langserver/providers/builtin_metadata.json`	Dev plaintext
`…/builtin_metadata.json.enc`	Encrypted blob for release
`…/builtin_metadata.json.sha256`	First 16 hex chars of SHA-256 of **plaintext**
`…/metadata_decrypt.py`	Fernet load + integrity check
`scripts/build/compile.py`	`encrypt_metadata()`, Nuitka include of providers dir
`scripts/build/ci_build.py`	`stage_metadata()` for CI
`scripts/build/.metadata.key`	**Gitignored** Fernet key material

Generation

After adding evaluator builtins:

```
python scripts/generate_builtin_metadata.py
```

Do not hand-edit the JSON as the long-term source of truth; regenerate from code.

Encryption (build)

`scripts/build/compile.py` → `encrypt_metadata()` / `_resolve_fernet_key()`:

1. Require `cryptography` and plaintext JSON.
2. Resolve key (first hit wins): env `CRYPTO_KEY` → `PYNESCRIPT_METADATA_KEY` → `METADATA_KEY` → existing `scripts/build/.metadata.key` → generate a new Fernet key and write that file (mode `0o600`).
3. Encrypt → `builtin_metadata.json.enc`.
4. Write truncated SHA-256 of plaintext to `.sha256`.

A random key is generated **only on first local encrypt** when no env/file key exists. For **byte-stable** encrypted artifacts across CI runs, supply a stable secret via the same env vars the encrypt path already reads (`CRYPTO_KEY`, `PYNESCRIPT_METADATA_KEY`, or `METADATA_KEY`), or reuse `scripts/build/.metadata.key`. Runtime decrypt uses a `.metadata.key` file next to the providers **or** env `PYNESCRIPT_METADATA_KEY`.

Without a stable key, each CI encrypt produces a different `.enc` blob — not a functional bug, but noisy diffs and cache misses.

Decrypt path

`metadata_decrypt.py`:

1. Resolve key: PyInstaller/Nuitka data dir `.metadata.key`, else providers dir, else `PYNESCRIPT_METADATA_KEY`.
2. Fernet-decrypt `.enc`.
3. If `.sha256` present, compare `sha256(plaintext).hexdigest()[:16]`.
4. `json.loads` → dict.

Both `get_metadata()` and `get_metadata_cached()` prefer **plaintext** when both artifacts exist (developer-friendly). Difference: `get_metadata()` returns `{}` on total failure; `get_metadata_cached()` raises `FileNotFoundError` if neither file is present.

Invariants and edge cases

1. **Regenerate after builtin changes** or completion/hover/inlay drift.
2. **Empty dict on failure** fails soft (no completions) rather than crashing the server.
3. **Integrity mismatch** raises in decrypt — compiled binary should not silently serve corrupt docs.
4. **Key not in git** — never commit `.metadata.key`.
5. **Fernet is symmetric** — anyone with the binary + key material can recover plaintext; design goal is casual reverse-engineering friction and future paid-metadata swap hooks.

Worked example — local dev

```
python scripts/generate_builtin_metadata.py
# edit-free check
python -c "from pynescript.langserver.providers.builtin_metadata import get_builtin; print(get_buil
```

Worked example — encrypted binary path

```
# build (encrypts + Nuitka) — see DevOps
python scripts/build/compile.py

# or embed .metadata.key in the onefile data dir during compile
```

Failure modes

Symptom	Cause
Completions empty in release binary	Missing key / wrong `PYNESCRIPT_METADATA_KEY`
`Metadata integrity check failed`	`.enc` and `.sha256` out of sync
CI churn on `.enc`	New random key each build without `CRYPTO_KEY` / `METADATA_KEY`
Stale docs for new `ta.*`	Forgot to re-run generator

See also

- Metadata crypto (DevOps)
- Nuitka build
- Completion
- Hover

VSCODE EXTENSION

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "VS Code Extension" description: "PYNE VS Code extension: LanguageClient host for pyne-lsp, TextMate grammar, snippets, settings, and VSIX 0.5.4."

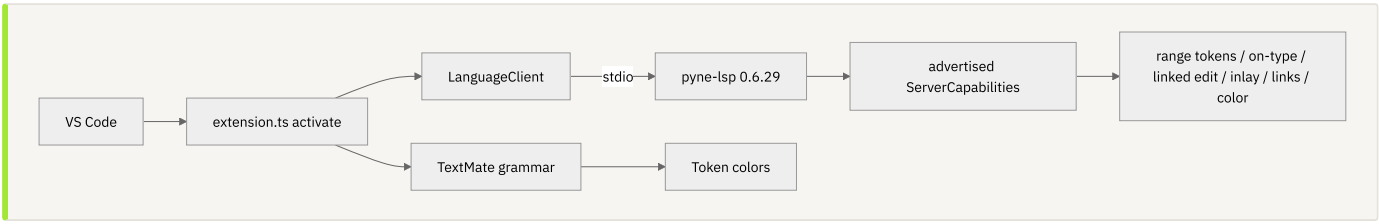
VS CODE EXTENSION

Abstract

PYNE (`hoox-sh.pyne` **0.5.4**) is a **LanguageClient host** plus a rich TextMate grammar, branded as part of the **HOOX open trading stack**. It does **not** reimplement language intelligence in TypeScript — `vscode-languageclient` speaks STDIO to `pyne-lsp` (then alias `pynescript-lsp`, then `python -m pynescript.langserver`) and forwards whatever that process advertises in `initialize`. The VSIX stays **0.5.4**; the hosted Python server is **`hoox-pyne`**

0.6.29 (`src/pynescript/__about__.py`). The extension maps **`.pyne` (first-class)** plus `*.pine` / `*.pinev5` / `*.pinev6` / `*.pinescript` to language id `pinescript` , ships **snippets**, and exposes a broad `pynescript.*` settings surface with per-feature toggles.

Conceptual model



Interface surface

Package identity

From `vscode-extension/package.json` :

Field	Value
name	`pyne`
displayName	`PYNE Language Support`
version	`0.5.4`
publisher	`hoox-sh`
extension id	`hoox-sh.pyne`
icon	`media/icon.png` (HOOX mark)
engines.vscode	`^1.91.0`
main	`./out/extension.js` (esbuild bundle)
dependency	`vscode-languageclient` ^10
galleryBanner	`#050505` dark

Language contribution

- **id:** `pinescript`
- **aliases:** `PYNE` , `pyne` , `pinescript` , `Pine`
- **extensions:** **`.pyne`** (first), `.pine` , `.pinev5` , `.pinev6` , `.pinescript`
- **configuration:** `language-configuration.json`
- **grammar:** `syntaxes/pinescript.tmLanguage.json` (`scopeName: source.pinescript`) — namespaces, annotations, hex colors, UDT/enum, multiline strings, plot/strategy builtins
- **snippets:** `snippets/pine.code-snippets` — `indicator` , `strategy` , `library` , `type` , `enum` , `input.*` , `plot` , `plotshape` , `ta.*` , `strategy.entry` / `close` , `if` / `for` / `switch` , `request.security` , `label.new` , `array.new<>` , multiline string

Hosted `pyne-lsp` methods (not reimplemented in TypeScript)

The LanguageClient negotiates capabilities from `src/pynescript/langserver/config.py` (`get_server_capabilities`). The hosted server now advertises the following; the 0.5.4 extension is still only a host, but adds client-side **feature gates** so each capability can be toggled without restarting the server.

LSP method	What the editor gets	Feature page
`textDocument/semanticTokens/full` + `/range`	Same legend, range clipped to the viewport	Semantic tokens
`textDocument/onTypeFormatting`	Format the just-completed line; trigger `"\n"`	Formatting
`textDocument/linkedEditingRange`	Same-document live ranges for a user ident (keywords / builtin namespaces refused)	Linked editing
`textDocument/inlayHint`	Display-only parameter-name hints on builtin calls (`ta.sma(close, 14)` → `source:` / `length:`). Hints never insert text	Inlay hints
`textDocument/documentLink` + `documentLink/resolve`	`http://` / `https://` URLs in comments (`//`, `/* */`, hash `#` …) and string literals. Resolve fills `tooltip` with the URL when tooltip is unset	Document links
`textDocument/documentColor` + `colorPresentation`	`#hex` (and named `color.*` / literal `color.rgb` / `color.new` / `color()` / `color.from_gradient`) swatches; presentation rewrites as `#rrggbb` / `#rrggbbaa`	Document color

Also advertised (unchanged host path): diagnostics, completion + resolve, hover, signature help, definition / references, highlight + rename, document / workspace symbols, full + range formatting, folding, selection range, Convert-to-v6 / W001 / W002 / C002 code actions. Capability table: Architecture.

`[pinescript]` configuration defaults already set `editor.semanticHighlighting.enabled` to `true` and `editor.defaultFormatter` to `hoox-sh.pyne`. Format-on-type and linked-editing UX are standard VS Code editor settings (`editor.formatOnType`, `editor.linkedEditing`) — they are **not** `pynescript.*` keys. Enable them if you want Enter to hit `onTypeFormatting` or rename-as-you-type to hit `linkedEditingRange`.

Activation

```
onLanguage:pinescript
onCommand:pynescript.restartServer
onCommand:pynescript.formatDocument
onCommand:pynescript.showLspOutput
onCommand:pynescript.showLspCommand
onCommand:pynescript.convertToV6
onCommand:pynescript.diagnosticsToggle
onCommand:pynescript.completionToggle
onCommand:pynescript.inlayHintsToggle
onCommand:pynescript.foldingToggle
onCommand:pynescript.colorPreviewToggle
onCommand:pynescript.toggleHover
onCommand:pynescript.toggleSignatureHelp
onCommand:pynescript.toggleSemanticTokens
onCommand:pynescript.toggleDocumentHighlight
onCommand:pynescript.toggleLinkedEditing
onCommand:pynescript.toggleSelectionRange
onCommand:pynescript.toggleDocumentLink
onCommand:pynescript.toggleCodeAction
onCommand:pynescript.toggleFormatOnSave
onCommand:pynescript.openDocumentLink
onCommand:pynescript.runOnPro
onCommand:pynescript.installLanguageServer
onCommand:pynescript.installBackend
onCommand:pynescript.startBackend
onCommand:pynescript.testBackend
onCommand:pynescript.selectSyntax
workspaceContains:**/*.pyne
workspaceContains:**/*.pine
workspaceContains:**/*.pinev5
workspaceContains:**/*.pinev6
```

Settings (`pynescript.*`)

Appearance

Key	Default	Meaning
`pynescript.syntaxHighlighting`	`void`	Pine token palette (TextMate + semantic tokens), independent of the workbench theme. `none` inherits the active theme.

Language server

Key	Default	Meaning
`pynescript.lsp.enabled`	`true`	Start the language client
`pynescript.lsp.command`	`auto`	`auto` (PATH → module), an absolute binary path, or `docker`
`pynescript.lsp.python`	`python3`	Interpreter for `-m pynescript.langserver`
`pynescript.lsp.args`	`[]`	Extra server arguments (`docker run` arguments when `command` is `docker`)

Editor features

Key	Default	Meaning
`pynescript.formatting.enabled`	`true`	Formatting — full document, selection, on-type
`pynescript.formatOnSave.enabled`	`false`	Format on save (requires `formatting.enabled`)
`pynescript.diagnostics.enabled`	`true`	Errors, warnings, and hints
`pynescript.diagnostics.severity`	`{}`	Remap severity by lint-code prefix, e.g. `{"W": "Hint"}`
`pynescript.diagnostics.style`	`false`	Show C001/C002/C004 style findings
`pynescript.diagnostics.semantic`	`false`	Opt-in UDT/enum member and user-function arity checks
`pynescript.completion.snippets`	`true`	Snippets for builtins and declarations
`pynescript.hover.enabled`	`true`	Hover cards (signature, docs, types)
`pynescript.inlayHints.parameters`	`true`	Parameter-name hints in calls
`pynescript.signatureHelp.enabled`	`true`	Parameter hints while typing a call
`pynescript.semanticTokens.enabled`	`true`	Semantic token colorization
`pynescript.documentHighlight.enabled`	`true`	Highlight other occurrences of an identifier
`pynescript.linkedEditing.enabled`	`true`	Rename-as-you-type (needs `editor.linkedEditing`)
`pynescript.selectionRange.enabled`	`true`	Smart expand/shrink selection
`pynescript.documentLink.enabled`	`true`	Clickable `http(s)` URLs in comments and strings
`pynescript.codeAction.enabled`	`true`	Code actions (Convert to v6, `W001` quick fix)

Feature settings are sent to the server through `workspace/didChangeConfiguration` and take effect without restarting the client. Only `pynescript.lsp.*` changes restart the language server.

Installation

Key	Default	Meaning
`pynescript.install.autoInstall`	`false`	Install the server with pip when missing, without prompting
`pynescript.install.autoPrompt`	`true`	Ask once before installing a missing server
`pynescript.install.package`	`hoox-pyne[lsp]`	pip requirement installed by **PYNE: Install Language Server**
`pynescript.install.python`	`""`	Python for pip (empty = `lsp.python` → `python3` → `python` → `py`)
`pynescript.install.args`	`[]`	Extra pip arguments (e.g. `["--no-cache-dir"]`)
`pynescript.install.createVenv`	`true`	Isolated venv when the system Python is PEP 668 managed
`pynescript.install.venv`	`""`	Explicit venv directory (empty = extension storage)

Pro & backend

Key	Default	Meaning
`pynescript.pro.enabled`	`false`	Enable PYNE Pro (`Run on Pro`)
`pynescript.pro.apiKey`	`""`	Pro API key, minted with `POST /auth/create_key` ; stored in settings in plain text
`pynescript.backend.url`	`http://127.0.0.1:5002`	Self-hosted backend base URL (no `/run`)
`pynescript.backend.useHosted`	`false`	Use `pynescript.online` instead of `backend.url`
`pynescript.backend.hostedUrl`	`https://pynescript.online`	Hosted backend base URL
`pynescript.backend.package`	`hoox-pyne[pro,lsp]`	pip requirement installed by **PYNE: Install Backend (Pro API)**
`pynescript.backend.path`	`""`	PYNE checkout for **PYNE: Start Backend** (empty = first workspace folder)

Themes (AXIS Void / Hoox Signal / PYNE Volt)

The extension contributes three color themes. Pick one from the normal VS Code **Preferences: Color Theme** picker — the extension never changes `workbench.colorTheme` automatically, not on activation and not on install.

Theme	Accent	Surfaces
AXIS Void	periwinkle `#939FFF`	Near-black with a whisper of indigo; dark-grey panels
Hoox Signal	signal orange `#FE6A00`	Near-black, neutral surfaces
PYNE Volt	lime `#B7EF09`	Near-black with a whisper of lime

Comments are always neutral gray in every theme. Keywords and functions take the accent, strings a light accent tint, numbers the foreground, and punctuation / series builtins gray — so a file stays calm even with a vivid accent.

Pine syntax highlighting is independent of the workbench theme:

Option	Setting	Values	`none` means
Syntax highlighting	`pynescript.syntaxHighlighting`	`none` · `void` · `signal` · `volt`	Use the active theme's Pine colors

It writes language-scoped `[pinescript]` token colors, so it composes with any workbench theme — including your own. Switch it with **PYNE: Select Syntax Highlighting** or set the key directly. Default: `void` .

Invariants and edge cases

1. **`lsp.enabled === false` skips starting the client** — output channel, status bar, and palette commands still register so the palette never shows “command not found”; restart then no-ops until the setting is true.
2. **Binary vs module:** production users should install `hoox-pyne[lsp]` or the Nuitka onefile (`pynescript-lsp-*` on GitHub Releases) and set `pynescript.lsp.command` if not on PATH.
3. **Grammar works without LSP** — open a `.pyne` or `.pine` file offline and still get TextMate highlighting.
4. **`.pyne` is first-class** in `package.json` extensions + activation; legacy `.pine*` / `.pinescript` remain fully supported.
5. File watcher covers every associated suffix (`pyne` , `pine` , `pinev5` , `pinev6` , `pinescript`).

6. **VSIX 0.5.4 does not pin server methods.** Range tokens, on-type format, linked editing, inlay resolve / param names, document links + resolve tooltip, and document color appear only when the spawned `pyne-lsp` is new enough to advertise them (this tree: **0.6.29**). Feature gates are client-side and read live settings.

Worked example — extension development

```
pip install -e ".[lsp]"
cd vscode-extension && npm ci && npm run compile
# Launch Extension Development Host pointing at vscode-extension
```

Open a fixture `.pine` file; confirm Problems panel shows parse/lint diagnostics from the Python server. With a current `pyne-lsp`, also expect `#hex` color swatches, clickable `https://` links in comments, `source:` / `length:` inlay labels on `ta.sma(close, 14)`, and range semantic tokens while scrolling.

Failure modes

Symptom	Cause
“Language client failed to start”	`pyne-lsp` missing / wrong command
No squiggles, highlighting OK	LSP disabled or server crash; check Output → PYNE Language Server
Restart does nothing	Client never started (`lsp.enabled` false)
No inlay param names / color swatches / document links	Spawned `pyne-lsp` is older than this tree (need **0.6.29**)
Enter does not reformat the previous line	`editor.formatOnType` is off (not a `pynescript.*` key)
Identifiers do not multi-cursor as you type	`editor.linkedEditing` is off (not a `pynescript.*` key)
A feature does nothing after toggling	Its `pynescript.<feature>.enabled` key is `false`
Run on Pro says “disabled”	Set `pynescript.pro.enabled` (and `pro.apiKey` for hosted)
Run on Pro request fails	Check `pynescript.backend.url` / `backend.useHosted` (base URL, no `/run`) and that the backend is reachable

See also

- Architecture — declared `ServerCapabilities`
- Semantic tokens — full and range
- Formatting — full, range, and on-type (`\n`)
- Linked editing
- Inlay hints — types, param names, resolve
- Document links — `http(s)` URLs; resolve fills tooltip
- Document color
- Clients — non-VS Code hosts
- Nuitka build
- Editors guide

CLIENTS

COPYRIGHT (C) 2024-2026 JANGO_BLOCKCHAINED
THIS FILE IS PART OF PYNESCRIPT.
PYNESCRIPT IS FREE SOFTWARE: YOU CAN REDISTRIBUTE IT AND/OR MODIFY
IT UNDER THE TERMS OF THE GNU AFFERO GENERAL PUBLIC LICENSE AS PUBLISHED BY
THE FREE SOFTWARE FOUNDATION, EITHER VERSION 3 OF THE LICENSE, OR
(AT YOUR OPTION) ANY LATER VERSION.
PYNESCRIPT IS DISTRIBUTED IN THE HOPE THAT IT WILL BE USEFUL,
BUT WITHOUT ANY WARRANTY; WITHOUT EVEN THE IMPLIED WARRANTY OF
MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. SEE THE
GNU AFFERO GENERAL PUBLIC LICENSE FOR MORE DETAILS.
YOU SHOULD HAVE RECEIVED A COPY OF THE GNU AFFERO GENERAL PUBLIC LICENSE
ALONG WITH PYNESCRIPT. IF NOT, SEE
HTTPS://WWW.GNU.ORG/LICENSES/.
SPDX-LICENSE-IDENTIFIER: AGPL-3.0-OR-LATER

title: "Editor Clients" description: "Wire pyne-lsp into Neovim, Zed, Emacs, Helix, Sublime, and generic LSP hosts."

EDITOR CLIENTS

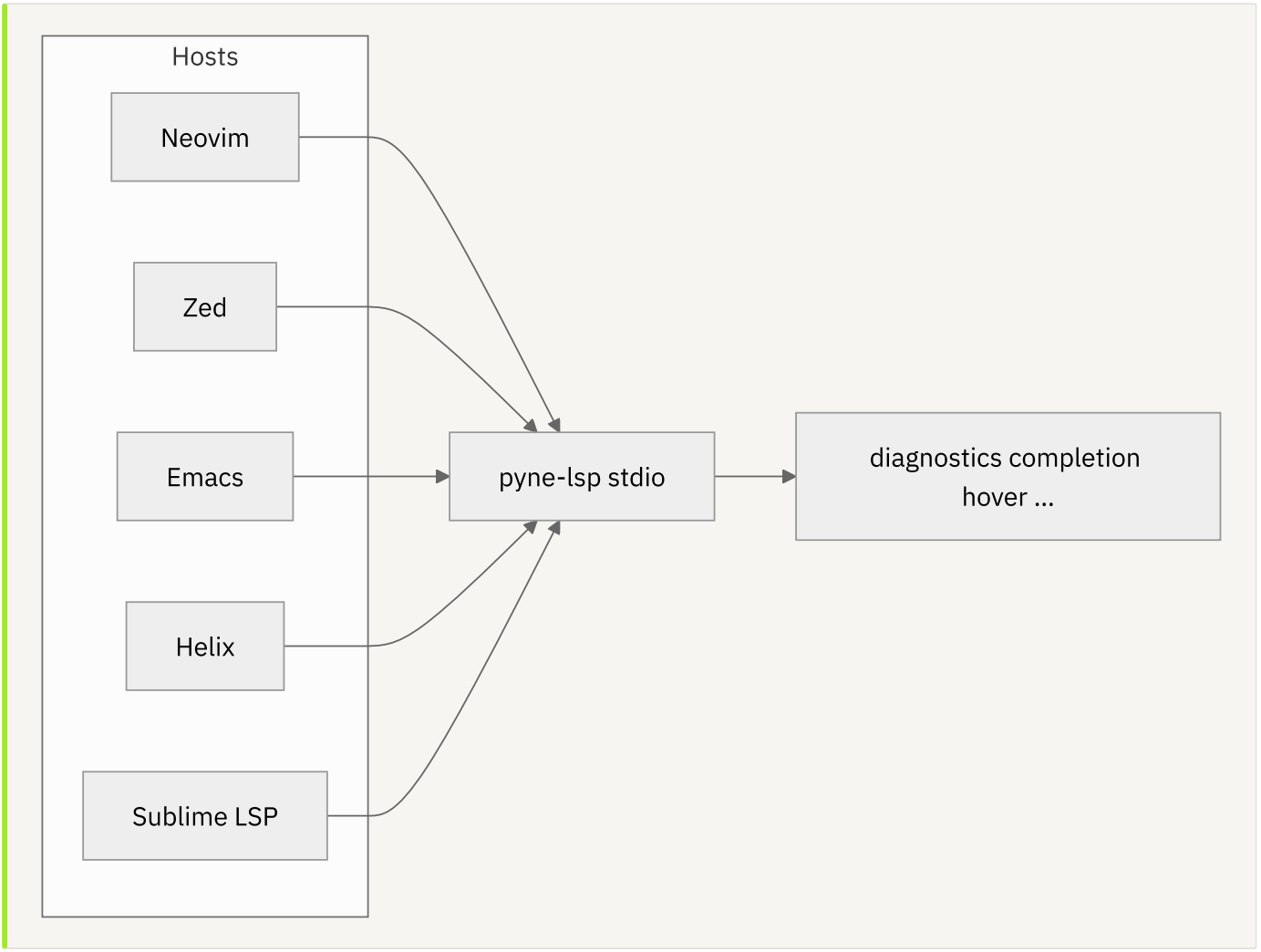
Abstract

Any editor that speaks LSP over STDIO can host PYNE intelligence. First-class snippets live in `clients/` (Neovim Lua, Zed JSON, Emacs Lisp) plus documented Helix/Sublime fragments. The VS Code path is a dedicated extension (vscode-extension); this page covers the rest.

Prerequisite: `pyne-lsp` (or `pynescript-lsp`) on `PATH` (Python 3.10+):

```
pip install "hoox-pyne[lsp]"
# development
pip install -e ".[lsp]"
```

Conceptual model



Interface surface — repo files

File	Editor
`clients/neovim.lua`	Neovim (`nvim-lspconfig` or manual)
`clients/zed.json`	Zed `settings.json` fragment
`clients/emacs.el`	Emacs `lsp-mode` + `pinescript-mode`
`clients/README.md`	Human-oriented install notes
`vscode-extension/`	VS Code / Cursor-compatible

Neovim

Return table from `clients/neovim.lua` :

```
return {
  cmd = { 'pyne-lsp' },
  filetypes = { 'pinescript' },
  root_dir = function(fname)
    return vim.fs.root(fname, { '.git', '*.pine', '*.pinev5', '*.pinev6', 'pyproject.toml' })
      or vim.fn.getcwd()
  end,
  settings = {
    pinescript = {
      formatting = { enabled = true },
      diagnostics = { enabled = true },
      completion = { snippets = true },
    },
  },
}
```

Suggested maps when attaching: `gd` definition, `gr` references, `K` hover, format via `vim.lsp.buf.format`.

Register filetype if needed:

```
au BufRead,BufNewFile *.pyne,*.pine,*.pinev5,*.pinev6,*.pinescript setfiletype pinescript
```

Zed

Merge `clients/zed.json` into `~/.config/zed/settings.json`:

```
{
  "languages": {
    "Pine Script": {
      "language_servers": ["pynescript"]
    }
  },
  "language_servers": {
    "pynescript": {
      "command": "pyne-lsp",
      "arguments": ["--stdio"],
      "languages": ["Pine Script"]
    }
  }
}
```

Zed does not ship a first-party Pine grammar in this repo. Map `*.pyne` / `*.pine` to the `"Pine Script"` language name (or whatever your grammar extension registers) so the `language_servers` key attaches.

Emacs

`clients/emacs.el` registers an `lsp-mode` client:

```
(lsp-register-client
 (make-lsp-client
  :new-connection (lsp-stdio-connection '("pyne-lsp" "--stdio"))
  :major-modes '(pinescript-mode)
  :server-id 'pynescript))
```

Defines a minimal `pinescript-mode` with font-lock keywords and `auto-mode-alist` for `\\.pine\\'` / `\\.pinev[0-9]+\\'` (add `\\.pyne\\'` locally if you use the product suffix). Optional keys: `C-c C-c` format, `M-.` definition, `M-?` references.

Helix

`~/.config/helix/languages.toml`:

```
[[language]]
name = "pinescript"
scope = "source.pinescript"
file-types = ["pyne", "pine", "pinev5", "pinev6"]
roots = ["pyproject.toml"]
command = "pyne-lsp"
args = ["--stdio"]
```

Sublime Text

With Package Control **LSP**:

```
{
  "clients": {
    "pynescript": {
      "command": ["pyne-lsp", "--stdio"],
      "selector": "source.pinescript",
      "initializationOptions": {}
    }
  }
}
```

Generic / Cursor

```
pyne-lsp
# aliases:
pynescript-lsp --stdio
# container (stdio):
docker run --rm -i -v "$PWD:/work" -w /work ghcr.io/hoox-sh/pyne/lsp:0.6.29
```

Point the host's "external language server" command at that binary. Cursor users can also load the VS Code extension (`hoox-sh.pyne`) in compatible mode when packaged as VSIX.

Internals

Server entry always uses STDIO (`start_io` in `__main__.py`). Capability negotiation is identical across hosts — differences are client UI only (how inlay hints render, whether pull diagnostics are used, etc.).

Tests: `tests/test_lsp_features.py` (handlers with fake params), `tests/test_langserver.py` (e2e with pytest-asyncio) — not editor-specific.

Invariants and edge cases

1. **Language id** should be `pinescript` for VS Code parity; some editors invent their own names — map filetypes carefully.
2. **Root directory** heuristics vary; wrong root rarely breaks single-file Pine editing (workspace is URI-keyed open buffers).
3. **Settings keys** under `pinescript` in Neovim are conventional; the Python server does not currently enforce a rich `workspace/configuration` schema for all of them.
4. Nuitka onefile can replace the module entry for air-gapped machines — same command name if installed on PATH.

Failure modes

Symptom	Cause
Server exits immediately	Missing pygls / incomplete install `.[lsp]`
Filetype never attaches	Extension not mapped to `pinescript`
Features missing in Helix	Older Helix without inlay / semantic support — grammar still useful

See also

- VS Code extension
- Architecture
- End-user editors guide